



Coinbase: Base Precompile Macros & Storage

Security Review

Cantina Managed review by:

J4X, Lead Security Researcher

Slowfi, Lead Security Researcher

Cryptara, Security Researcher

Jay, Security Researcher

August 14, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Native <code>SSTORE</code> path omits EIP-2200 gas-stipend guard	4
3.1.2	Generic precompile success conversion drops accumulated <code>SSTORE</code> refunds	5
3.1.3	<code>set_code</code> bypasses <code>STATICCALL</code> immutability enforcement	6
3.1.4	Dynamic storage shrink-overwrites leave stale tail slots	7
3.1.5	Storage layout hash derivation bypasses metered keccak accounting	9
3.2	Low Risk	12
3.2.1	<code>VecHandler</code> indexing allows out-of-bounds storage writes	12
3.2.2	Stateful precompiles ignore the EIP-8037 state gas reservoir	13
3.2.3	<code>SSTORE</code> omits state-gas accounting while <code>set_code</code> retains it	15
3.2.4	Existing-account <code>set_code</code> skips EIP-8037 code-deposit state gas	15
3.2.5	Transient <code>Set<T></code> operations panic instead of returning structured errors	18
3.2.6	<code>SSTORE</code> Mutates State Before Gas Is Charged	18
3.2.7	<code>with_account_info</code> passes an <code>AccountInfo</code> whose <code>code</code> field is never hydrated	19
3.2.8	Production checkpoint commits ignore saved checkpoint tokens	20
3.2.9	<code>HashMap</code> checkpoint rollback omits production-relevant state	21
3.2.10	Slot offset arithmetic can overflow or alias slots	22
3.2.11	<code>install(address)</code> override can desynchronize dispatch and storage addresses	23
3.2.12	Namespace metadata constants can collide with contract field	25
3.2.13	Case-normalized field names can generate duplicate slot constants	27
3.3	Informational	30
3.3.1	EIP-3541 comment documents the wrong <code>set_code</code> behavior	30
3.3.2	Unused / unreachable code in the B20-asset and activation precompiles	32
3.3.3	<code>StorageKey</code> rustdoc incorrectly documents divergent mapping key encodings as Solidity-compatible	33
3.3.4	Packing module rustdoc uses stale <code>Layout::Slot</code> and misstates fixed-array layout behavior	34
3.3.5	Generated store logic carries stale comments referencing nonexistent upstream code	35
3.3.6	Mapping-Only Storable Structs Emit Invalid <code>IS_DYNAMIC</code>	36
3.3.7	Decimals Default Uses a Magic Literal Instead of the Named Bound	36
3.3.8	Dead Code in Generated <code>role_admin</code> Default Fallback Branch	37
3.3.9	Namespace macro contract branch is unreachable under documented usage	37
3.3.10	<code>install</code> Parser Reports a Misleading Error for Unsupported Options	38
3.3.11	<code>with_caller</code> uses a redundant manual <code>drop</code> and restores the caller through a panicking borrow	39
3.3.12	Contract macro diagnostic omits supported address attribute	39
3.3.13	Precompile <code>install</code> diagnostic omits supported <code>addr</code> attribute	40
3.3.14	Stablecoin decimals Fallback Returns 0 Instead of the System Default	41
3.3.15	<code>[slot(key)]</code> computes a non ERC 7201 slot and misleads on namespacing	42
3.3.16	Release builds do not enforce generated storage collision checks	42
3.3.17	<code>args()</code> empty-list duplicate is not rejected	43
3.3.18	<code>#[contract]</code> silently drops all non- <code>derive</code> struct-level attributes	44
3.3.19	<code>STATICCALL</code> state-change violations are returned as ordinary reverts	44
4	Appendix	46
4.1	Unit Test Coverage Improvements	46
4.1.1	<code>crates/common/precompile-storage/src/evm.rs</code>	46
4.1.2	<code>crates/common/precompile-storage/src/provider.rs</code>	48
4.1.3	<code>crates/common/precompile-macros/src/utils.rs</code>	49

4.1.4	<code>crates/common/precompile-macros/src/accounting.rs</code>	51
4.1.5	<code>crates/common/precompile-macros/src/storable.rs</code>	52
4.1.6	<code>crates/common/precompile-macros/src/contract.rs</code>	52
4.1.7	<code>crates/common/precompile-macros/src/namespace.rs</code>	53

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Jun 1st to Jun 9th the Cantina team conducted a review of [base](#) on commit hash [6ee3da63](#). The team identified a total of **37** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	5	3	2
Low Risk	13	3	10
Gas Optimizations	0	0	0
Informational	19	12	7
Total	37	18	19

3 Findings

3.1 Medium Risk

3.1.1 Native `SSTORE` path omits EIP-2200 gas-stipend guard

Severity: Medium Risk.

Context: `evm.rs#L172-L189`.

Summary: Base native precompile storage writes do not enforce EIP-2200's `gasleft <= 2300` `SSTORE` guard before mutating storage. As a result, low-gas direct calls to stateful native precompiles can perform warm dirty storage writes that the normal EVM `SSTORE` opcode would halt before executing.

Description: EIP-2200 requires an `SSTORE` to fail the current call frame with out-of-gas when the frame's remaining gas is less than or equal to the call stipend. The pinned `revm` opcode implementation applies this sentry before host storage mutation: it checks `gas.remaining() <= call_stipend()` and halts with `ReentrancySentry00G ; CALL_STIPEND` is 2300 gas in the same dependency set.

The native precompile wrapper does not mirror that opcode-level guard. The stateful wrapper only rejects non-direct calls, copies calldata, constructs `EvmPrecompileStorageProvider`, and enters `StorageCtx` :

```
if !input.is_direct_call() {
    return ::base_precompile_storage::BasePrecompileError::revert(
        ::base_precompile_storage::DelegateCallNotAllowed {},
    )
    .into_precompile_result(0, 0);
}

let mut provider = ::base_precompile_storage::EvmPrecompileStorageProvider::new(
    input,
    ::revm::context_interface::cfg::GasParams::default(),
);
```

`crates/common/precompiles/src/macros.rs#L6-L23` .

ABI dispatchers then deduct calldata gas and forward into the precompile logic without checking the stipend boundary, for example B20 asset dispatch at `crates/common/precompiles/src/b20_asset/dispatch.rs#L22-L52` , policy dispatch at `crates/common/precompiles/src/policy/dispatch.rs#L18-L27` , activation dispatch at `crates/common/precompiles/src/activation/dispatch.rs#L16-L27` , and factory dispatch at `crates/common/precompiles/src/b20_factory/dispatch.rs#L15-L19` .

The actual native storage provider only blocks static context, calls `internals.sstore(address, key, value)` , and charges `SSTORE` gas afterward:

```
fn sstore(&mut self, address: Address, key: U256, value: U256) -> Result<()> {
    if self.is_static {
        return Err(BasePrecompileError::StaticCallViolation);
    }
    let s = self
        .internals
        .sstore(address, key, value)
        .map_err(|e| BasePrecompileError::Fatal(e.to_string()))?;

    self.deduct_gas(self.gas_params.sstore_static_gas());
    self.deduct_gas(self.gas_params.sstore_dynamic_gas(true, &s.data, s.is_cold));
    self.refund_gas(self.gas_params.sstore_refund(true, &s.data));

    Ok(())
}
```

`crates/common/precompile-storage/src/evm.rs#L172-L189` .

This is reachable through normal direct precompile calls. For example, B20 `approve` takes `ctx.caller()` as the allowance owner and calls `self.approve(caller, c.spender, c.amount)` at `crates/common/precompiles/src/b20_asset/dispatch.rs#L191-L194`. `approve` then writes the allowance and emits `Approval` at `crates/common/precompiles/src/common/ops/transferable.rs#L109-L120`, and the allowance field is a direct mutable mapping at `crates/common/precompiles/src/common/core_storage.rs#L33-L36`.

A transaction that first warms and dirties the same allowance slot can later make a low-gas direct call into `approve`. If the remaining gas is at or below 2300 but still enough for the warm dirty rewrite and subsequent event accounting, the native precompile path can mutate storage where the canonical opcode path would have halted before the write.

Impact Explanation: Impact is Medium. The issue breaks an EVM semantic guarantee that contracts rely on as a reentrancy sentry: stipend-level calls should not be able to perform `SSTORE`. The affected path is limited to native precompile storage, so this is not arbitrary EVM storage mutation, but it can still allow state changes in Base native precompiles under gas conditions that EIP-2200 intentionally forbids.

Likelihood Explanation: Likelihood is Medium. Attackers need a direct call into a stateful native precompile with a target slot already warm and dirty in the same transaction, because cold or expensive writes are expected to run out of gas after the mutation attempt. Those preconditions are realistic for allowance-style B20 flows and other repeated writes to the same precompile slot, and the outer wrapper does not otherwise reject low-gas direct calls.

Proof of Concept:

Recommendation: We recommend enforcing the EIP-2200 stipend sentry before every native precompile `sstore` mutation. `EvmPrecompileStorageProvider::sstore` should check the current frame's remaining gas before `internals.sstore` and return an out-of-gas halt when remaining gas is less than or equal to `gas_params.call_stipend()`. Add regression coverage for a warm dirty B20 allowance write where a normal `SSTORE` would halt at the stipend boundary.

Coinbase: Fixed in [PR 3285](#).

Cantina Managed: Fix verified.

3.1.2 Generic precompile success conversion drops accumulated `SSTORE` refunds

Severity: Medium Risk.

Context: [error.rs#L142-L154](#).

Summary: Successful native precompile calls that mutate storage through the generic dispatch result path can drop accumulated EIP-3529 `SSTORE` refunds, causing callers to overpay gas for otherwise valid refund-producing operations.

Description: The storage context already has a refund-preserving success constructor: `storage_ctx.rs` builds a `PrecompileOutput` and then copies `self.gas_refunded()` into `out.gas_refunded`. The production EVM storage provider also records refunds for refundable `SSTORE`s: `evm.rs` charges storage write gas and calls `refund_gas(...)`, while `evm.rs` records and exposes the accumulated refund.

However, the generic `IntoPrecompileResult` success conversion does not preserve that field:

```
impl<T> IntoPrecompileResult<T> for Result<T> {
    fn into_precompile_result(
        self,
        gas: u64,
        state_gas: u64,
        encode_ok: impl FnOnce(T) -> Bytes,
    ) -> PrecompileResult {
```

```

    match self {
      Ok(res) => Ok(PrecompileOutput::new(gas, encode_ok(res), state_gas)),
      Err(err) => err.into_precompile_result(gas, state_gas),
    }
  }
}

```

This means any accumulated `ctx.gas_refunded()` is lost when a mutating precompile returns through the generic success arm at `error.rs`.

This is reachable in current production dispatchers, not just a future API hazard. Affected dispatchers include:

- `PolicyRegistryStorage::dispatch`, which executes write calls and then returns through `result.into_precompile_result(...)` at `dispatch.rs`.
- `ActivationRegistryStorage::dispatch` at `dispatch.rs`.
- `B20FactoryStorage::dispatch` at `dispatch.rs`.
- `B20AssetStorage::dispatch_with_observer` at `dispatch.rs`.
- `B20StablecoinStorage::dispatch_with_observer` at `dispatch.rs`.

Several normal successful operations can clear previously nonzero storage before reaching those generic success returns. Examples include:

- Clearing a pending policy admin in `storage.rs`.
- Deleting the pending admin during finalization or renounce at `storage.rs`.
- Deactivating a feature at `storage.rs`.
- Zeroing balances through B20 transfers at `transferable.rs`.
- Zeroing allowances through B20 approvals at `transferable.rs`.
- Full-balance or full-supply burns at `burnable.rs`.

The broken guarantee is that gas refunds accumulated by the storage provider should be propagated into the precompile output so revm can apply them to transaction-level refund accounting under the EIP-3529 cap. Instead, these successful paths can return `gas_refunded == 0` even after the provider recorded a valid refund.

Recommendation: We recommend making the generic success path preserve accumulated refunds. One approach is to replace dispatcher success returns with a `StorageCtx`-aware constructor equivalent to `StorageCtx::success_output`, or extend `IntoPrecompileResult::into_precompile_result` to accept and copy the current refund counter into `PrecompileOutput.gas_refunded`. Add regression tests for at least one dispatcher call that clears a nonzero storage slot and assert that the returned `PrecompileOutput.gas_refunded` matches the refund accumulated by the provider.

Coinbase: Fixed in [PR 3258](#).

Cantina Managed: Fix verified.

3.1.3 `set_code` bypasses `STATICCALL` immutability enforcement

Severity: Medium Risk.

Context: (No context files were provided by the reviewer).

Summary: `set_code` mutates account bytecode without checking whether the current precompile execution is static.

Description: The EVM storage provider rejects static execution for ordinary state-changing operations, but `set_code` does not apply the same guard. The guarded sibling mutators are:

- `sstore` at `evm.rs`.
- `tstore` at `evm.rs`.
- Event emission.

In `evm.rs`, `set_code` performs gas accounting and then calls `internals.set_code(...)`.

The in-memory provider mirrors the same behavior: `hashmap.rs` updates `code_hash` and `code` without checking `is_static`. Generated precompile initialization reaches this path through `layout.rs`, which installs sentinel bytecode through `self.storage.set_code(self.address, bytecode)?`.

This breaks the EVM guarantee that a `STATICCALL` context cannot perform state-changing side effects. Any static-context path that reaches code installation can mutate account bytecode even though surrounding callers expect immutable execution.

Relevant code:

```
fn set_code(&mut self, address: Address, code: Bytecode) -> Result<()> {
    let code_len = code.len();
    self.deduct_gas(self.gas_params.code_deposit_cost(code_len))?;
    // no self.is_static check before mutation
    self.internals
        .set_code(address, code)
        .map_err(|e| BasePrecompileError::Fatal(e.to_string()))
}

fn sstore(&mut self, address: Address, key: U256, value: U256) -> Result<()> {
    if self.is_static {
        return Err(BasePrecompileError::StaticCallViolation);
    }
    // ...
}
```

Recommendation: We recommend adding the same static-context guard before any gas charging or code mutation in both storage providers. The guard should match the existing behavior for:

- `sstore`
- `tstore`
- `emit_event`

```
if self.is_static {
    return Err(BasePrecompileError::StaticCallViolation);
}
```

Coinbase: Fixed in [PR 3252](#).

Cantina Managed: Fix verified.

3.1.4 Dynamic storage shrink-overwrites leave stale tail slots

Severity: Medium Risk.

Context: (No context files were provided by the reviewer).

Summary: Dynamic storage writers overwrite only the new logical value and do not clear storage slots that belonged to a previous longer `Bytes`, `String`, or `Vec<T>` value.

Description: Dynamic bytes-like values and vectors use a length field to bound normal typed reads, but the storage writers do not clear retired backing slots when a shorter value replaces a longer one. As a result, obsolete chunks or elements remain in raw storage after the overwrite.

This same issue was already fixed upstream in Tempo in [PR 3840](#), `fix(storage): cleanup tail in dyn types`, which was merged on May 7, 2026. The Tempo PR describes the same root cause: overwriting a dynamic storable (`Vec<T>`, `String`, or `Bytes`) with a shorter value left stale tail slots populated, and the fix clears stale tails on shrinking writes.

For `Bytes` and `String`, `store_bytes_like()` writes the new base slot and only the chunks required by the new byte length. It never loads the previous encoded length and therefore cannot clear old chunks beyond the new payload.

[types/bytes_like.rs#L212-L233](#).

```
fn store_bytes_like<S: StorageOps>(bytes: &[u8], storage: &mut S, base_slot: U256) ->
↳ Result<()> {
    let length = bytes.len();
    if length <= 31 {
        storage.store(base_slot, encode_short_string(bytes))
    } else {
        storage.store(base_slot, encode_long_string_length(length))?;
        let slot_start = calc_data_slot(base_slot);
        let chunks = calc_chunks(length);

        for i in 0..chunks {
            let slot = slot_start + U256::from(i);
            /* writes only the new chunks */
            storage.store(slot, U256::from_be_bytes(chunk_bytes))?;
        }

        Ok(())
    }
}
```

The delete path shows that these long-form chunks are part of the value footprint and are expected to be zeroed when removed:

[types/bytes_like.rs#L236-L249](#).

```
fn delete_bytes_like<S: StorageOps>(storage: &mut S, base_slot: U256) -> Result<()> {
    let base_value = storage.load(base_slot)?;
    let is_long = is_long_string(base_value);

    if is_long {
        let length = calc_string_length(base_value, true)?;
        let slot_start = calc_data_slot(base_slot);
        let chunks = calc_chunks(length);
        for i in 0..chunks {
            storage.store(slot_start + U256::from(i), U256::ZERO)?;
        }
    }

    storage.store(base_slot, U256::ZERO)
}
```

The same pattern exists for vectors. `Vec<T>::store()` updates the length and writes only the new prefix. Packed vectors write only `calc_packed_slot_count(new_len)` slots, and unpacked vectors iterate only the new elements.

[types/vec.rs#L59-L72](#).

```
fn store<S: StorageOps>(&self, storage: &mut S, len_slot: U256, ctx: LayoutCtx) ->
↳ Result<()> {
    debug_assert_eq!(ctx, LayoutCtx::FULL, "Dynamic arrays cannot be packed");
```

```

storage.store(len_slot, U256::from(self.len()))?;
if self.is_empty() {
    return Ok(());
}

let data_start = calc_data_slot(len_slot);
if T::BYTES <= 16 {
    store_packed_elements(self, storage, data_start, T::BYTES)
} else {
    store_unpacked_elements(self, storage, data_start)
}
}

```

By contrast, `Vec<T>::delete()` loads the existing length and clears every old packed slot or unpacked element, confirming that ordinary overwrite skips cleanup that deletion performs.

`types/vec.rs#L75-L98` .

```

let length = load_checked_len(storage, len_slot)?;
storage.store(len_slot, U256::ZERO)?;

if T::BYTES <= 16 {
    let slot_count = calc_packed_slot_count(length, T::BYTES);
    for slot_idx in 0..slot_count {
        storage.store(data_start + U256::from(slot_idx), U256::ZERO)?;
    }
} else {
    for elem_idx in 0..length {
        let elem_slot = data_start + U256::from(elem_idx * T::SLOTS);
        T::delete(storage, elem_slot, LayoutCtx::FULL)?;
    }
}

```

Typed reads are length-bounded, so the stale tail is masked when callers read the same type normally. However, the obsolete data remains visible through raw storage access and storage inspection, storage refunds for clearing removed values are missed, and future growth or manual slot use can encounter data that callers would reasonably expect to have been removed when assigning a shorter dynamic value.

Recommendation: We recommend making dynamic `store()` clear any previously used tail when the new value is shorter than the old value. Bytes-like writers should load the prior encoded length before overwrite and zero old chunks beyond the new chunk count, including long-to-short transitions. Vector writers should load the prior length and clear the range `new_len..old_len`; packed vectors should zero retired packed slots, while unpacked vectors should call `T::delete` for retired elements. If needed for efficiency, add an explicit zero-initialized storage context so first writes and appends can skip the old-length read safely.

Coinbase: Acknowledged.

Cantina Managed: An initial fix was submitted in [PR 3384](#), but it does not yet cover `bytes_like.rs` or auto-generated field setters. The fields with a reachable post-initialization mutation path are `name`, `symbol`, and `contract_uri` on `B20CoreStorage`, all of which are updatable by accounts with the `Metadata` role. Typed reads remain correct because they are always length-bounded; stale bytes are only visible via raw slot inspection (`eth_getStorageAt`). The `currency` field on `B20StablecoinStorage` is set at initialization only and is not exposed as a mutable ABI function, so it is unaffected in practice.

3.1.5 Storage layout hash derivation bypasses metered keccak accounting

Severity: Medium Risk.

Context: (No context files were provided by the reviewer).

Summary: Runtime storage layout helpers compute mapping and dynamic-data slots with direct `keccak256` calls instead of the gas-charging storage-context hash API. These hashes correspond to

storage address derivations that Solidity/EVM bytecode would normally execute through the `KECCAK256` opcode, so precompile calls can force internal slot-derivation hashes without paying the corresponding runtime keccak gas.

Description: The storage provider exposes a metered hash primitive. The default provider implementation computes the EVM keccak price and deducts gas before hashing:

```
/// Computes keccak256 and charges the appropriate gas.
fn keccak256(&mut self, data: &[u8]) -> Result<B256> {
    let num_words =
        u64::try_from(data.len().div_ceil(32)).map_err(|_|
            ↪ BasePrecompileError::OutOfGas)?;
    let price = KECCAK256WORD
        .checked_mul(num_words)
        .and_then(|w| w.checked_add(KECCAK256))
        .ok_or(BasePrecompileError::OutOfGas)?;
    self.deduct_gas(price)?;
    Ok(keccak256(data))
}
```

`provider.rs`.

However, the runtime storage-layout helpers bypass that metered API. `StorageKey::mapping_slot` derives every mapping element slot by calling `alloy_primitives::keccak256` directly after building the 64-byte Solidity mapping preimage:

```
fn mapping_slot(&self, slot: U256) -> U256 {
    let key_bytes = self.as_storage_bytes();
    let key_bytes = key_bytes.as_ref();
    debug_assert!(key_bytes.len() <= 32);

    let mut buf = [0u8; 64];
    buf[32 - key_bytes.len()..32].copy_from_slice(key_bytes);
    buf[32..].copy_from_slice(&slot.to_be_bytes::<32>());

    U256::from_be_bytes(keccak256(buf).0)
}
```

`provider.rs`.

`MappingHandler` reaches this helper whenever a key handler is materialized, including mutable access used by write paths:

```
pub fn at_mut(&mut self, key: &K) -> &mut V::Handler<'a>
where
    K: StorageKey + Eq + Clone + Ord,
{
    let (base_slot, address, storage) = (self.base_slot, self.address, self.storage);
    self.cache.get_or_insert_mut(key, || {
        V::handle(key.mapping_slot(base_slot), LayoutCtx::FULL, address, storage)
    })
}
```

`mapping.rs`.

The same pattern exists for dynamic array data bases. `VecHandler` computes `self.data_slot()` before pushing or indexing elements, but `calc_data_slot` directly hashes the length slot and cannot charge gas because it has no storage context:

```
let mut elem_slot =
    Self::compute_handler(self.data_slot(), self.address, self.storage, length);
elem_slot.write(value)?;
```

```
#[inline]
pub(crate) fn calc_data_slot(len_slot: U256) -> U256 {
```

```
U256::from_be_bytes(keccak256(len_slot.to_be_bytes::<32>()).0)
}
```

`vec.rs` and `vec.rs` .

Bytes-like dynamic values and string mapping keys also use direct hashes for their long-data base or string-key mapping preimage:

```
fn mapping_slot(&self, slot: U256) -> U256 {
    let mut buf = Vec::with_capacity(self.len() + 32);
    buf.extend_from_slice(self.as_bytes());
    buf.extend_from_slice(&slot.to_be_bytes::<32>());
    U256::from_be_bytes(keccak256(buf).0)
}
```

```
fn calc_data_slot(base_slot: U256) -> U256 {
    U256::from_be_bytes(keccak256(base_slot.to_be_bytes::<32>()).0)
}
```

`bytes_like.rs` and `bytes_like.rs` .

This is reachable from externally controlled precompile inputs. For example, policy membership updates accept an address array, cap it at 64 accounts, and then perform two nested mapping derivations for each account: one for `policy_id` and one for the account under that policy mapping.

```
Self::require_account_batch_size(accounts)?;
for account in accounts {
    if add {
        self.members.at_mut(&policy_id).at_mut(account).write(true)?;
    } else {
        self.members.at_mut(&policy_id).at_mut(account).delete()?;
    }
}
```

`storage.rs` .

The dispatch wrapper only charges a separate per-calldata-word fee, using a hardcoded `G_SHA3WORD = 6` ; it does not account for the internal mapping or dynamic-data slot hashes performed after ABI decoding:

```
macro_rules! deduct_calldata_cost {
    ($ctx:expr, $calldata:expr $(,)?) => {{
        const G_SHA3WORD: u64 = 6;

        let calldata_len = $calldata.len();
        let calldata_cost = calldata_len.div_ceil(32).saturating_mul(G_SHA3WORD as
        ↪ usize) as u64;
        if let Err(e) = $ctx.deduct_gas(calldata_cost) {
            return e.into_precompile_result($ctx.gas_used(), $ctx.state_gas_used());
        }
    }};
}
```

`macros.rs` .

As a result, validators perform extra host keccak work that is absent from the reported gas usage. Storage access gas still applies to the eventual `SLOAD` or `SSTORE` , but these runtime layout hashes are separate computation and the code already contains a metered keccak primitive for that exact class of work. Calls that touch many distinct mapping keys or dynamic elements therefore undercharge CPU work relative to the number of keccak derivations they force.

This issue is limited to runtime hashes that replace storage address derivations a Solidity contract would normally perform with the `KECCAK256` opcode. Compile-time macro hashes, such as literal `#[slot("...")]` or namespace-root derivation performed during procedural macro expansion, are not part of this claim because they do not execute during a precompile call and would not be EVM-metered at runtime.

Recommendation: We recommend routing storage-layout hash derivation through the metered provider path. For example, add context-aware helpers for mapping-slot and dynamic-data-slot derivation that call `StorageCtx::keccak256`, then use those helpers from `MappingHandler`, `VecHandler`, bytes/string storage, and any generated accessors that materialize dynamic handlers.

Where repeated derivations are unavoidable, cache the derived slot after the first charged computation within the handler. Also replace the hardcoded calldata word charge with a value sourced from the active gas parameters, or document and enforce why calldata-copy charging is intentionally independent from EVM keccak pricing.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. The extra gas cost of unmetered keccak slot derivation is small relative to the cost of `SSTORE` or `SLOAD`, and the long-term plan is for native precompile execution costs to be lower than standard EVM costs.

3.2 Low Risk

3.2.1 `VecHandler` indexing allows out-of-bounds storage writes

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Summary: `VecHandler` bounds-checks `at()`, but its indexing operators derive writable element handlers for any index without checking the vector length.

Description: `VecHandler::at()` treats the stored vector length as part of the collection boundary: it reads `len_slot` and returns `None` when `index >= len`.

The `Index` and `IndexMut` implementations do not enforce that boundary. Both implementations compute and cache an element handler directly from `data_slot` and the requested index without reading `len_slot`. A caller can therefore access `handler[index]` for an index outside the logical vector and then call `write()` on the returned handler.

That write stores data at the slot derived for the out-of-bounds index, but it does not update the vector length. The value is unreachable through normal vector reads, can later become visible if the vector grows to that index, and violates the expected behavior of Rust's indexing operator, which should not silently provide mutable access outside the collection bounds.

Affected code:

- `crates/common/precompile-storage/src/types/vec.rs#L204-L215`.
- `crates/common/precompile-storage/src/types/vec.rs#L257-L278`.

Impact Explanation: Impact is Low. This is a storage correctness issue that can create unreachable or stale data in contract storage, but no direct asset-loss path was established from the scoped code.

Likelihood Explanation: Likelihood is Medium. The unsafe path is exposed through normal Rust indexing syntax on a public handler type, while the safe `at()` helper exists but is not enforced by the type system.

Recommendation: We recommend making the indexing operator enforce the same bounds as `at()`. Since `Index` cannot return a `Result`, either make `Index` and `IndexMut` panic on out-of-bounds access after reading `len_slot`, or remove these trait implementations and require callers to use `at()` and `push()` APIs that can return storage errors.

Coinbase: Fixed in PR 3207.

Cantina Managed: Fix verified.

3.2.2 Stateful precompiles ignore the EIP-8037 state gas reservoir

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Summary: Stateful native precompiles charge EIP-8037 state-gas through regular gas and return it as reservoir gas, so post-EIP-8037 account/code creation through `set_code` can be overcharged and misaccounted.

Description: EIP-8037 splits gas accounting into regular gas and state gas. State-gas charges should consume the state-gas reservoir first and only spill into regular gas after the reservoir is exhausted. The upstream revm/alloy path already models this separation: precompile results are converted with `Gas::new_with_regular_gas_and_reservoir(gas_limit, output.reservoir)`, and revm's gas model documents that state-gas charges deduct from the reservoir before spilling into regular gas.

The stateful precompile storage provider drops the reservoir before any precompile storage operation can use it. `EvmPrecompileStorageProvider::new` deconstructs `PrecompileInput` with `..`, discarding `input.reservoir`, and initializes a regular-only gas tracker:

```
pub fn new(input: PrecompileInput<'a>, gas_params: GasParams) -> Self {
    let PrecompileInput { gas, caller, is_static, internals, .. } = input;

    Self {
        internals,
        caller,
        gas: Gas::new(gas),
        gas_params,
        is_static,
        state_gas_used: 0,
        // ...
    }
}
```

`set_code` then applies EIP-8037 state-gas charges when it installs code for a new account:

```
if is_new_account {
    self.deduct_gas(self.gas_params.create_cost());
    let num_words = code_len.div_ceil(32) as u64;
    self.deduct_gas(KECCAK256.saturating_add(KECCAK256WORD.saturating_mul(num_words)));
    self.deduct_state_gas(self.gas_params.create_state_gas());
    self.deduct_state_gas(self.gas_params.code_deposit_state_gas(code_len));
}
```

However, `deduct_state_gas` always spends regular gas and never consults a reservoir. The provider also reports that no reservoir exists:

```
fn deduct_state_gas(&mut self, gas: u64) -> Result<()> {
    // No separate reservoir in the precompile context; state gas is drawn from regular
    ↪ gas.
    self.deduct_gas(gas)?;
    self.state_gas_used = self.state_gas_used.saturating_add(gas);
    Ok(())
}

fn reservoir(&self) -> u64 {
    0
}
```

The B20 factory reaches this path during token creation. It creates a fresh token account and calls `self.storage.set_code(token_address, stub)?`, so post-EIP-8037 B20 token creation performs account/code state-gas accounting through this regular-gas-only provider.

The return path also swaps the state-gas and reservoir semantics. `IntoPrecompileResult` passes `state_gas` as the third argument to `PrecompileOutput::new`:

```
Ok(res) => Ok(PrecompileOutput::new(gas, encode_ok(res), state_gas)),
```

But `PrecompileOutput::new(gas_used, bytes, reservoir)` interprets that third argument as the remaining reservoir and initializes `state_gas_used` to zero. Therefore successful stateful precompile calls can return regular gas that already includes state-gas charges, set the returned reservoir to the amount of state gas used, and leave the actual `state_gas_used` field unset. This is inconsistent with the EIP-8037 reservoir model and with the revm/alloy precompile result contract.

Impact Explanation: Impact is Medium. Affected stateful precompile calls can consume regular gas for costs that should be absorbed by the EIP-8037 state-gas reservoir, which can reject otherwise valid calls with insufficient regular gas or overstate regular gas consumption. The same calls can also underreport `state_gas_used` while returning the used state gas as remaining reservoir gas, causing block-level regular/state gas accounting to diverge from the intended EIP-8037 model. No direct asset-loss path was established, so the impact is not High.

Likelihood Explanation: Likelihood is Medium. The bug is deterministic for stateful native precompile paths that perform state creation with a nonzero EIP-8037 reservoir. B20 token creation reaches the affected `set_code` path for fresh token accounts. The condition depends on post-EIP-8037 execution where a nonzero reservoir is available to the precompile call, so it is not assigned High likelihood for all current executions.

Proof of Concept: The issue follows from the existing accounting path:

1. Execute a post-EIP-8037 stateful precompile call with nonzero `PrecompileInput.reservoir`.
2. Reach B20 token creation, which calls `self.storage.set_code(token_address, stub)?` for a fresh account.
3. `EvmPrecompileStorageProvider::new` discards the input reservoir and creates `Gas::new(gas)`.
4. `set_code` calls `deduct_state_gas(create_state_gas)` and `deduct_state_gas(code_deposit_state_gas(code_len))`.
5. Each `deduct_state_gas` call invokes `deduct_gas`, reducing regular gas instead of consuming the reservoir first.
6. The successful result is returned through `PrecompileOutput::new(gas, bytes, state_gas)`, which treats `state_gas` as remaining reservoir and leaves `state_gas_used` unset.

A correct implementation would leave regular gas unchanged while the reservoir is sufficient, reduce the reservoir by the state-gas cost, and return the consumed amount through `PrecompileOutput.state_gas_used`.

Recommendation: We recommend preserving the precompile input reservoir in `EvmPrecompileStorageProvider` and initializing the provider gas tracker with the same regular-gas/reservoir split used by revm. `deduct_state_gas` should use reservoir-first semantics equivalent to revm's `record_state_cost`, and `reservoir()` should return the actual remaining reservoir.

We recommend fixing precompile result construction so regular gas used, state gas used, and remaining reservoir are separate values. Success and revert helpers should pass the remaining reservoir to `PrecompileOutput::new` or `PrecompileOutput::revert`, then explicitly assign `out.state_gas_used = ctx.state_gas_used()` before returning.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. There are no immediate plans to support EIP-8037, so this fix is deferred.

3.2.3 `SSTORE` omits state-gas accounting while `set_code` retains it

Severity: Low Risk.

Context: [evm.rs#L172-L189](#).

Description: Base's precompile storage layer tracks two gas quantities: regular gas for computational cost, and state gas for the EIP-8037 permanent storage burden. State gas is surfaced from every precompile dispatch via `state_gas_used()` in the returned `PrecompileOutput` and flows upward into block-level accounting through `block_state_gas_used()`.

The provider charges state gas asymmetrically. The bug location is the `sstore` method of `EvmPrecompileStorageProvider`:

`set_code`, the factory contract-creation path, charges `create_state_gas` plus `code_deposit_state_gas`. `Sstore`, every storage-slot write performed by a B20 precompile, charges no state gas at all. For a cold zero to non-zero write the dropped charge is 230,000 gas, the dominant component of the operation's true cost of roughly 252,200. The existing test suite covers only the `set_code` state-gas path, so no test exercises or pins `SSTORE` state gas, and the omission goes undetected.

This leaves a state that is neither fully on nor fully off, and exactly one of two interpretations must be true:

1. If `EIP-8037` state metering is meant to be active for B20 precompiles, which requires precompiles to use the same gas path as the EVM, then `sstore` is wrong. Every precompile-driven storage-slot creation escapes state-gas accounting. If a per-block state-growth limit is enforced using this counter, B20 `SSTORE`s that create new slots are not counted against it, allowing a block to create more persistent state than the limit intends. This is a state-bloat and underpricing risk.
2. If state metering is meant to be inactive for these precompiles, consistent with Base hardcoding the reservoir to 0 and documenting that state gas is drawn from regular gas, then `set_code` is wrong. It should not be charging state gas either, and `state_gas_used` should be uniformly zero.

The evidence leans toward an accidental omission. State gas is charged in `set_code` but not in `sstore`, while the `state_gas_used` counter remains fully wired into block accounting, and no test pins `SSTORE` state gas.

Recommendation: Make state-gas accounting uniform across `sstore` and `set_code`. The half-on, half-off state must not ship. If metering is intended to be on, restore the `SSTORE` state-gas charge by adding `self.deduct_state_gas(self.gas_params.sstore_state_gas(&s.data))?` after the dynamic-gas deduction. If metering is intended to be off, remove the `deduct_state_gas` calls from `set_code` so `state_gas_used` is consistently zero.

If the current divergence is intentional, it would be ideal to document it, as an intentional behavioral divergence from the upstream gas model that is invisible in the code is itself a defect.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. This fix is deferred as there are no immediate plans to support EIP-8037.

3.2.4 Existing-account `set_code` skips EIP-8037 code-deposit state gas

Severity: Low Risk.

Context: [evm.rs#L96-L119](#).

Summary: `set_code` only charges EIP-8037 code-deposit state gas when the target account is completely empty. A user can precompute their deterministic B20 token address, send 1 wei to that address before token creation, and then call the factory. The 1 wei transfer makes the future token account existent while leaving its code empty, so the factory still accepts it but the later precompile stub code deposit skips the required `L * CPSB` state-gas charge.

Description: EIP-8037 separates account-creation state gas from code-deposit state gas. For `CREATE / CREATE2` with bytecode size `L`, it charges `L * CPSB` when code is deposited into an already existent account, and `(STATE_BYTES_PER_NEW_ACCOUNT + L) * CPSB` only when the account leaf is new. The EIP also states that an account with nonzero balance but no code and zero nonce is existent, and that contract creation at that address should charge only the code-deposit component.

The B20 factory gives users a direct way to create this condition. A caller controls the factory salt, and the token address is deterministically derived from `(caller, variant, salt)`. Before calling `createB20`, the caller can compute the future token address and send 1 wei to it. That transfer creates a balance-only account leaf at the future token address. The account still has an empty code hash, so it passes the factory's deployment check, but it is no longer `AccountInfo::is_empty()`.

The production precompile storage provider does not preserve that distinction. It first loads the account and checks `AccountInfo::is_empty()`, then charges both EIP-8037 state-gas components only inside that single `is_new_account` branch:

`crates/common/precompile-storage/src/evm.rs#L89-L119`.

```
let is_new_account = {
    let state_load = self
        .internals
        .load_account(address)
        .map_err(|e| BasePrecompileError::Fatal(e.to_string()))?;
    state_load.data.info.is_empty()
};

if is_new_account {
    self.deduct_state_gas(self.gas_params.create_state_gas())?;
    self.deduct_state_gas(self.gas_params.code_deposit_state_gas(code_len))?;
}
```

The in-memory provider mirrors the same behavior, so tests built on it also treat every non-empty account leaf as exempt from code-deposit state gas:

`crates/common/precompile-storage/src/hashmap.rs#L89-L100`.

The reachable production path is B20 token creation. `B20FactoryStorage::create_b20` computes a deterministic token address, rejects only accounts with non-empty code, and then writes a one-byte `0xef` stub through `set_code`:

`crates/common/precompiles/src/b20_factory/storage.rs#L59-L71`.

```
let already_deployed =
    self.storage.with_account_info(token_address, |info|
        → Ok(!info.is_empty_code_hash()))?;
if already_deployed {
    return Err(BasePrecompileError::revert(IB20Factory::TokenAlreadyExists {
        token: token_address,
    }));
}

let checkpoint = self.storage.checkpoint();
let stub = Bytecode::new_legacy(Bytes::from_static(&[0xef]));
self.storage.set_code(token_address, stub)?;
```

Therefore, a user can save gas on B20 token creation with this sequence:

1. Choose the `variant` and `salt` that will be used for `createB20`.
2. Compute the deterministic token address derived from `(caller, variant, salt)`.
3. Send 1 wei to that future token address before creation.
4. Call `createB20` with the same `variant` and `salt`.

The factory accepts the address because the account still has no code. However, because the account now has a nonzero balance, `AccountInfo::is_empty()` is false. `EvmPrecompileStorageProvider::`

`set_code` then skips both `create_state_gas()` and `code_deposit_state_gas(1)`. Skipping `create_state_gas()` is correct for the existing account leaf; skipping `code_deposit_state_gas(1)` is not.

Impact Explanation: The impact is Low. The issue is a protocol gas-accounting undercharge rather than unauthorized state mutation, asset loss, or corruption of token balances. A caller can intentionally pay 1 wei to the future token address to make creation cheaper by avoiding the EIP-8037 code-deposit state-gas charge. In the currently reachable B20 factory path, the deposited bytecode is the one-byte `0xef` marker, so the missed EIP-8037 charge is bounded to one `CPSB` unit. With the Amsterdam `CPSB` value used by the local gas parameters, that is 1,530 state gas per affected code deposit.

Likelihood Explanation: The likelihood is Medium. The path is user-reachable through B20 factory creation because callers control the salt and can precompute the resulting token address. Creating the required state only requires sending 1 wei to that address before calling the factory. It is not expected to affect every token creation because fresh, never-touched addresses still take the `is_new_account` branch and pay both state-gas components.

Proof of Concept: The attack path is:

1. The caller chooses a `salt` and `variant`.
2. The caller computes the future token address using the same deterministic derivation as `B20Variant::compute_address(caller, salt)`.
3. The caller transfers 1 wei to that future address, creating an existent account with empty code.
4. The caller invokes `createB20` with the chosen `salt` and `variant`.
5. `create_b20` checks only `!info.is_empty_code_hash()`, so the prefunded empty-code account passes.
6. `set_code` sees `AccountInfo::is_empty() == false` and skips `code_deposit_state_gas(1)`.

The repository already contains focused tests demonstrating the two relevant provider branches:

```
crates/common/precompile-storage/src/evm.rs#L305-L341.
```

Run:

```
RUSTFLAGS='' cargo test -p base-precompile-storage
↳ set_code_existing_account_skips_state_gas -- --nocapture
RUSTFLAGS='' cargo test -p base-precompile-storage
↳ set_code_new_account_charges_create_and_deposit_state_gas -- --nocapture
```

Both tests pass. The first confirms that the current provider deliberately leaves `state_gas_used()` unchanged after `set_code` is called on an already-initialized account. The second confirms that a brand-new account is charged `create_state_gas() + code_deposit_state_gas(code_len)`. Together with the B20 factory's non-empty-code-only guard, this reproduces the missing code-deposit state-gas component for the 1 wei prefunded empty-code account.

Recommendation: We recommend separating the account-leaf creation charge from the code-deposit charge in `set_code`. Keep `create_state_gas()` gated on `AccountInfo::is_empty()`, but charge `code_deposit_state_gas(code_len)` whenever non-empty code is written to an account whose current code hash is empty, including existent balance-only accounts. The `HashMapStorageProvider` should mirror the same rule so tests exercise the production accounting model.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. There are no immediate plans to support EIP-8037, so this fix is deferred.

3.2.5 Transient `Set<T>` operations panic instead of returning structured errors

Severity: Low Risk.

Context: [set.rs#L306-L315](#).

Summary: `SetHandler<T>` transient methods panic even though the handler trait returns `Result`.

Description: The shared `Handler<T>` trait defines transient operations as `Result`-returning methods at [provider.rs](#):

- `t_read`
- `t_write`
- `t_delete`

`SetHandler<T>` implements those methods with `unimplemented!()` at [set.rs](#).

A generic caller using the handler trait can therefore trigger a panic instead of receiving a structured `BasePrecompileError` or unsupported-operation error. `Set` and `SetHandler` are publicly exported from the storage library, making this a public API hazard.

Relevant code:

```
pub trait Handler<T: Storable> {
    fn t_read(&self) -> Result<T>;
    fn t_write(&mut self, value: T) -> Result<()>;
    fn t_delete(&mut self) -> Result<()>;
}

fn t_write(&mut self, _: Set<T>) -> Result<()> {
    unimplemented!("Set does not support transient storage")
}
```

Recommendation: We recommend replacing the `unimplemented!()` stubs with explicit structured errors indicating that transient storage is unsupported for `Set<T>`.

Coinbase: Fixed in PR 3281.

Cantina Managed: Fix verified.

3.2.6 `SSTORE` Mutates State Before Gas Is Charged

Severity: Low Risk.

Context: [evm.rs#L172-L189](#).

Description: `sstore` calls `internals.sstore()`, which dirties and warms the slot in the revm journal **before** charging the static and dynamic `SSTORE` gas. If either `deduct_gas` returns `OutOfGas`, the slot is already written and the provider relies entirely on an outer call-frame rollback to undo it. The provider is not locally atomic on the out-of-gas path, and affordability is checked only after the state transition used to derive the cost.

Today this is masked because Base's stateful precompiles run inside a frame that revm reverts on `OutOfGas`. That protection is incidental and undocumented: any direct `StorageProvider` use, or any future intrinsic/handler-invoked path without a surrounding frame, would observe a persisted write for an operation that should have failed.

Tempo carried the identical `sstore` and fixed it:

- [tempoxyz/tempo PR 2321](#) — labeled `C-Bug` / `C-audit`; description: *"gas was deducted after mutating state. If an OOG error occurred mid-operation, partial state changes were already committed, leaving the chain in an inconsistent state."*

- **tempoxyz/tempo PR 2329** — merged fix (`C-Bug` / `C-audit`) for the observed consequence: a transaction that ran out of gas left a `key_authorization` write persisted — proving the “outer frame always reverts” assumption does not hold for every path.

Recommendation: Charge `SSTORE` gas before applying the journal mutation, matching the Tempo fix: compute the cold/warm classification, deduct the static cost (and confirm the cold load is affordable), then write. Base is a new chain with no legacy blocks, so this can be done unconditionally. If revm only exposes the classification through a mutating call, wrap the mutation in a `StorageCtx` checkpoint and revert it before returning `OutOfGas`. Add a regression test asserting a gas-failed `sstore` leaves the slot unchanged and unwarmed.

Coinbase: Fixed in PR 3310.

Cantina Managed: Fix verified.

3.2.7 `with_account_info` passes an `AccountInfo` whose `code` field is never hydrated

Severity: Low Risk.

Context: [evm.rs#L126-L149](#).

Description: `with_account_info` calls `load_account` and clones the resulting `AccountInfo` into the callback. `load_account` populates `code_hash` but leaves `code` lazy, so the `AccountInfo` handed to the callback can carry `code: None` even when the account holds real bytecode.

The upstream tempo implementation hydrates the code with `account.load_code()` before invoking the callback, so the bytes are always present:

```
account.load_code()?;  
  
f(&account.data.account().info);
```

Upstream reference: [tempoxyz/tempo crates/precompiles/src/storage/evm.rs](#).

The Base implementation omits that `load_code` call, which is the deviation. The two production callers, `is_initialized` and the B20 factory existence check, only read `is_empty_code_hash`, which depends on `code_hash` and is correctly populated. No current caller dereferences `code`, so the bug is latent rather than exploitable. Any future caller, or external consumer of this public trait, that reads `info.code` would silently observe missing or empty bytecode and behave incorrectly. Omitting the load also skips the cold code load gas that upstream charges, so the precompile may undercharge relative to upstream in some cases.

Recommendation: Hydrate the code before reading the info, matching upstream behavior, so `code` is always present in the callback and the cold code load gas is charged. Force the load on the account before cloning:

```
let (info, is_cold) = {  
    let mut state_load = self  
        .internals  
        .load_account(address)  
        .map_err(|e| BasePrecompileError::Fatal(e.to_string()))?;  
    state_load.data.load_code()?;  
    (state_load.data.info.clone(), state_load.is_cold)  
};
```

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. No caller currently reads `info.code` — all three callers (`is_initialized`, `create_b20_with_observer`, `is_b20_initialized`) only inspect `code_hash`, which is eagerly populated by `load_account`. The decision not to call `load_code` is now documented in [evm.rs](#) via PR 3386.

3.2.8 Production checkpoint commits ignore saved checkpoint tokens

Severity: Low Risk.

Context: [evm.rs#L265-L272](#).

Summary: The production EVM storage provider ignores the checkpoint token supplied by `CheckpointGuard::commit()` and commits whichever checkpoint is currently at the top of the underlying journal stack.

Description: `StorageCtx::checkpoint()` returns a guard containing a `JournalCheckpoint`, and `CheckpointGuard::commit()` passes that saved token back to the storage provider. This makes the public API appear token-based: committing a guard should commit that guard's checkpoint.

In the production EVM provider, however, the checkpoint argument is intentionally ignored. `checkpoint_commit()` calls `internals.checkpoint_commit()`, whose behavior is to pop the current top checkpoint rather than the supplied token.

[evm.rs#L261-L268](#).

```
fn checkpoint(&mut self) -> JournalCheckpoint {
    self.internals.checkpoint()
}

fn checkpoint_commit(&mut self, _checkpoint: JournalCheckpoint) {
    // alloy-evm's checkpoint_commit pops the top checkpoint; the arg is unused.
    self.internals.checkpoint_commit();
}
```

The test provider enforces the intended LIFO discipline instead. Its `checkpoint_commit()` asserts that the supplied checkpoint is the latest snapshot before popping it.

[hashmap.rs#L211-L218](#).

```
fn checkpoint_commit(&mut self, checkpoint: JournalCheckpoint) {
    assert_eq!(
        checkpoint.journal_i,
        self.snapshots.len() - 1,
        "out-of-order checkpoint commit (expected top of stack)"
    );
    self.snapshots.pop();
}
```

This creates a backend mismatch. If nested checkpoint guards are resolved out of order, the production backend commits the latest checkpoint even when the caller commits an older guard. The wrong checkpoint can be committed while the older guard remains active, and a later drop/revert can apply to a different state boundary than the caller intended. The HashMap backend catches this misuse, but production EVM execution does not, so tests can only detect the issue when they exercise the HashMap provider directly.

The same LIFO expectation applies to revert. The production backend forwards the token to `checkpoint_revert()`, but there is no local assertion that the reverted checkpoint is the active top-of-stack checkpoint.

[evm.rs#L270-L272](#).

```
fn checkpoint_revert(&mut self, checkpoint: JournalCheckpoint) {
    self.internals.checkpoint_revert(checkpoint);
}
```

Recommendation: We recommend enforcing LIFO checkpoint usage in `EvmPrecompileStorageProvider` before committing or reverting. A debug-only stack of active checkpoint identifiers is sufficient to catch API misuse during tests and debug builds while preserving release behavior. The guard should track checkpoint creation and assert that each commit or revert resolves the most recently created checkpoint. Alternatively, redesign the checkpoint API so out-of-order guard resolution is unrepresentable.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. The `JournalCheckpoint` parameter was removed from `PrecompileStorageProvider::checkpoint_commit`, but `CheckpointGuard::commit()` still does not validate that the guard being committed is the active top-of-stack checkpoint. As a result, committing an older guard while a newer nested guard is live still commits the current top checkpoint rather than the saved checkpoint. The `HashMap` backend's commit-side `checkpoint.journal_i == snapshots.len() - 1` assertion was also removed, so the test backend no longer catches out-of-order checkpoint commits. Enforcing the LIFO invariant in the `CheckpointGuard` wrapper, or making out-of-order guard commits unrepresentable, remains an open follow-up.

3.2.9 `HashMap` checkpoint rollback omits production-relevant state

Severity: Low Risk.

Context: [hashmap.rs#L204-L209](#).

Summary: The `HashMap` storage provider snapshots persistent storage slots and events, but omits transient storage and account/code state that should participate in checkpoint rollback.

Description: `HashMapStorageProvider::Snapshot` currently stores only `internals` and `events` at [hashmap.rs](#). The checkpoint implementation snapshots only those fields at [hashmap.rs](#).

Current snapshot coverage:

Field	Currently snapshotted?	Should rollback on checkpoint revert?	Rationale
<code>internals</code>	Yes	Yes	Persistent storage writes made after a checkpoint should be reverted.
<code>events</code>	Yes	Yes	Logs emitted inside reverted execution should be discarded.
<code>transient</code>	No	Yes	Transient storage writes made after a checkpoint should be reverted if the checkpoint is reverted.
<code>accounts</code>	No	Yes	<code>set_code</code> mutates account bytecode after checkpoint creation in factory-style flows, and that code/account state should disappear if initialization reverts.
<code>caller</code>	No	No	Caller is call-context state; the only runtime mutation path is <code>with_caller</code> , which uses a guard to restore the previous caller on scope exit, including early returns.
<code>is_static</code>	No	No	Static mode is call-context configuration and is only mutated in the <code>HashMap</code> backend through test setup helpers.
<code>counter_sload</code> / <code>counter_sstore</code>	No	No	These are test instrumentation for attempted operations, and attempts still occurred even if the state changes are reverted.
<code>gas_params</code>	No	No	Gas parameters are configuration for the call/test setup, not journaled EVM state.
<code>state_gas_used</code> and other gas accounting	No	No	Gas and state-gas accounting represent resources consumed during attempted execution; failed or reverted execution normally still burns gas.

Public provider methods mutate omitted state:

- `set_code` updates account bytecode at [hashmap.rs](#).

- `tstore` updates transient storage at `hashmap.rs`.

These state changes are not restored by HashMap checkpoint rollback even though the production EVM journal should roll them back.

Relevant code:

```
struct Snapshot {
    internals: HashMap<Address, U256>, U256>,
    events: HashMap<Address, Vec<LogData>>,
}

fn checkpoint(&mut self) -> JournalCheckpoint {
    self.snapshots
        .push(Snapshot { internals: self.internals.clone(), events: self.events.clone()
            ↪ });
    // ...
}
```

Recommendation: We recommend expanding `Snapshot` to include the omitted mutable fields that are expected to participate in checkpoint atomicity:

- Transient storage.
- Account/code state.

Do not roll back static mode, gas parameters, gas/state-gas accounting, or operation counters. Those fields should remain outside checkpoint atomicity, and the provider API or tests should document that boundary explicitly.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. This finding was downgraded to Low severity because the impact is limited to the `HashMap` test backend; the production EVM journal handles rollback correctly.

3.2.10 Slot offset arithmetic can overflow or alias slots

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Summary: Several storage handlers derive child slots with unchecked or saturating `U256` arithmetic instead of consistently returning `SlotOverflow`.

Description: The storage helpers derive child storage slots by adding offsets to a base slot. The crate already defines `BasePrecompileError::SlotOverflow`, and generated struct-array code uses `checked_add()` and returns that error when slot arithmetic exceeds `U256::MAX`.

Other scoped type paths do not follow the same rule:

- `Slot::new_at_offset()` and `Slot::new_at_loc()` use `saturating_add()`, so different overflowing offsets can alias to `U256::MAX`.
- Primitive array code uses unchecked `+` on `U256`.
- `VecHandler` uses unchecked `+` on `U256`.
- Bytes-like data chunk loops use unchecked `+` on `U256`.
- `SetHandler` uses unchecked `+` on `U256`.
- The default `Storable::delete()` path uses unchecked `+` on `U256`.

The `ruint` implementation backing Alloy `U256` implements `Add` as `wrapping_add`, so overflowed slot derivation can wrap to low storage slots instead of being rejected.

This makes slot derivation inconsistent across the storage abstraction. A layout rooted near `U256::MAX`, whether by explicit slot selection, namespace root, or derived dynamic data slot, can produce wrapped or saturated child slots and potentially read or write a different storage location than intended.

Affected code:

- `slot.rs`.
- `array.rs`.
- `vec.rs`.
- `bytes_like.rs`.
- `provider.rs`.
- `storable_primitives.rs`.

Relevant code:

```
slot: base_slot.saturating_add(U256::from_limbs([offset_slots as u64, 0, 0, 0])),  
  
let slot = if T::BYTES <= 16 {  
    data_start + U256::from(location.offset_slots)  
} else {  
    data_start + U256::from(index * T::SLOTS)  
};  
  
for offset in 0..Self::SLOTS {  
    storage.store(slot + U256::from(offset), U256::ZERO?);  
}
```

Recommendation: We recommend using checked slot arithmetic consistently for every derived storage slot and returning `BasePrecompileError::SlotOverflow` on failure. This should include:

- `Slot::new_at_offset()`
- `Slot::new_at_loc()`
- Array handlers.
- Vector handlers.
- Bytes-like chunk loops.
- Set position base derivation.
- The default multi-slot delete path.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. [PR 3415](#) replaces several affected paths with checked arithmetic, including `Slot::new_at_offset`, `Slot::new_at_loc`, `Vec`, bytes-like chunk loops, and `Set` position derivation. However, the primitive fixed-array storage generated by `precompile-macros` (`storable_primitives.rs`) and the default multi-slot `Storable::delete` path in `provider.rs` still use unchecked `U256` addition and remain unaddressed.

3.2.11 `install(address)` override can desynchronize dispatch and storage addresses

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Summary: The `#[precompile]` macro allows an `install(address = ...)` override to register a pre-compile at one address while generated dispatch still constructs storage bound to the storage type's canonical contract address.

Description: The `#[precompile]` macro parses `install(address = ...)` / `install(addr = ...)` as an arbitrary install address expression. When present, that expression is used as the key inserted into the EVM `PrecompilesMap`:

```
let install = config.install.map(|install| {
    let address = install
        .address
        .map_or_else(|| quote! { <#storage>::ADDRESS }, |address| quote! { #address });

    quote! {
        pub fn install(
            precompiles: &mut ::alloy_evm::precompiles::PrecompilesMap,
            #(#install_arg_defs),*
        ) {
            precompiles.extend_precompiles(::core::iter::once((
                #address,
                Self::precompile(#(#install_arg_names),*),
            )));
        }
    }
});
```

`precompile.rs#L43-L60` .

The generated runtime wrapper does not pass that install address into the storage object. It always constructs storage through `<Storage>::new(ctx)`:

```
pub fn precompile(#(#arg_defs),*) -> ::alloy_evm::precompiles::DynPrecompile {
    #macro_path!(#id, |ctx, calldata| {
        <#storage>::new(ctx).dispatch(ctx, &calldata #(<#arg_names>*)
    })
}
```

`precompile.rs#L71-L75` .

The storage constructor generated by `#[contract(addr = ...)]` binds the contract address into `new()` and stores it in `self.address`. Subsequent initialization and event emission use `self.address`, not the precompile-map key used by `install(address = ...)`:

```
pub fn new(storage: ::base_precompile_storage::StorageCtx<'a>) -> Self {
    Self::__new(#addr, storage)
}

fn __initialize(&mut self) -> ::base_precompile_storage::Result<()> {
    let bytecode = ::revm::state::Bytecode::new_legacy(::alloy_primitives::Bytes::from_s(
        ↪ tatic(&[0xef]));
    self.storage.set_code(self.address, bytecode)?;
    Ok(())
}

fn emit_event(&mut self, event: impl ::alloy_primitives::IntoLogData) ->
    ↪ ::base_precompile_storage::Result<()> {
    self.storage.emit_event(self.address, event.into_log_data())
}
```

`layout.rs#L150-L191` .

As a result, a precompile declared with mismatched values such as `#[precompile(install(address = A))]` and `#[contract(addr = B)]` can be installed and called at address `A`, while the generated storage, bytecode marker, and event emission operate under address `B`.

Current scoped usage did not show a concrete in-repository `#[precompile(install(address = ...))]` instantiation with divergent addresses. The issue is therefore an integration hazard in the macro API: the macro accepts an unsafe configuration without enforcing that dispatch identity and storage identity remain the same.

Recommendation: We recommend rejecting mismatched install and storage addresses at macro expansion time, or removing the independent install-address override. If an override is still needed, thread the chosen install address into the generated storage constructor so that the precompile-map key, storage address, bytecode marker, and emitted-event address all derive from the same value.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. No current in-repository usage has divergent install and storage addresses, so there is no live bug. A compile-time guard will be added to the macro to reject mismatched addresses as a follow-up.

3.2.12 Namespace metadata constants can collide with contract field

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Summary: Contract-level storage namespaces emit fixed metadata constants into the generated `slots` module, but the `#[contract]` field parser does not reserve the source field names that normalize to those constants. A contract field named `namespace_id` or `namespace_root` therefore makes the macro generate duplicate `NAMESPACE_ID` or `NAMESPACE_ROOT` constants and prevents the storage layout from compiling.

This is a Low severity issue: it can block developers from defining otherwise valid namespace-bearing storage layouts, but the observed impact is compile-time denial of use rather than runtime asset loss.

Description: The `#[contract]` macro rejects only three exact field names: `address`, `storage`, and `msg_sender`. The comparison is performed against the original source identifier string, before the identifier is normalized into the uppercase constant name used in the generated `slots` module.

`crates/common/precompile-macros/src/contract.rs#L37-L37` .

```
pub(crate) const RESERVED: &[&str] = &["address", "storage", "msg_sender"];
```

`crates/common/precompile-macros/src/contract.rs#L106-L110` .

```
if RESERVED.contains(&name.to_string().as_str()) {
    return Err(syn::Error::new_spanned(
        name,
        format!("Field name '{name}' is reserved"),
    ));
}
```

Field slot constants are generated from the field name by uppercasing it. For example, a field named `namespace_id` becomes `NAMESPACE_ID`.

`crates/common/precompile-macros/src/packing.rs#L40-L42` .

```
pub(crate) fn const_name(name: &Ident) -> String {
    name.to_string().to_uppercase()
}
```

When a contract-level namespace is present, the macro emits namespace metadata constants and the field constants into the same generated `slots` module.

`crates/common/precompile-macros/src/layout.rs#L243-L250` .

```
quote! {
    /// Storage slot indices and packing constants for this contract.
    pub mod slots {
        use super::*;
```

```
#namespace_constants
#constants
```

`crates/common/precompile-macros/src/layout.rs#L259-L265` .

```
quote! {
    /// ERC-7201 namespace identifier for this contract storage layout.
    pub const NAMESPACE_ID: &str = #id;

    /// ERC-7201 namespace root slot for this contract storage layout.
    pub const NAMESPACE_ROOT: ::alloy_primitives::U256 =
```

As a result, this otherwise normal namespace-bearing layout cannot compile:

```
#[contract(addr = TEST_ADDR)]
#[namespace("collision.test")]
pub struct NamespaceCollision {
    pub namespace_id: U256,
}
```

The field passes the reserved-name check because `"namespace_id"` is not in `RESERVED`, but field constant generation then emits `pub const NAMESPACE_ID: U256 = ...` into the same module that already contains `pub const NAMESPACE_ID: &str = ...` for the namespace metadata. The same root cause applies to `namespace_root`, which normalizes to `NAMESPACE_ROOT`.

During validation, the related case-sensitive `AddRes` / `ADDRESS` hypothesis was not reproduced as a collision with a separate macro-generated reserved constant in `slots` ; the confirmed issue is the namespace metadata collision.

Proof of Concept:

1. Check out the audited repository at commit `6ee3da6325e7416812065c58358274a086f9f223`.
2. From any working directory outside the repository, create a minimal Rust crate that depends on the local macro and storage crates:

```
export BASE_REPO=/path/to/base
mkdir -p base-macro-collision-poc/src
cat > base-macro-collision-poc/Cargo.toml <<EOF
[package]
name = "base-macro-collision-poc"
version = "0.1.0"
edition = "2024"

[dependencies]
alloy-primitives = { version = "1.5.6", default-features = false }
base-precompile-macros = { path = "$BASE_REPO/crates/common/precompile-macros" }
base-precompile-storage = { path = "$BASE_REPO/crates/common/precompile-storage",
↳ features = ["test-utils"] }
revm = { version = "38.0.0", default-features = false }
EOF
```

3. Add a namespace-bearing contract layout with a field named `namespace_id` :

```
cat > base-macro-collision-poc/src/lib.rs <<'EOF'
use alloy_primitives::{Address, U256, address};
use base_precompile_macros::contract;

const TEST_ADDR: Address = address!("00000000000000000000000000000000000000000000000000000000000000001234");

#[contract(addr = TEST_ADDR)]
#[namespace("collision.test")]
pub struct NamespaceCollision {
    pub namespace id: U256,
```

```
}  
EOF
```

4. Compile the reproduction crate:

```
RUSTFLAGS= CARGO_ENCODED_RUSTFLAGS= cargo check --manifest-path  
↳ base-macro-collision-poc/Cargo.toml
```

The command fails with the duplicate generated constant:

```
error[E0428]: the name `NAMESPACE_ID` is defined multiple times  
--> src/lib.rs:7:5  
|  
7 |     #[contract(addr = TEST_ADDR)]  
|     ~~~~~ `NAMESPACE_ID` redefined here  
|  
= note: `NAMESPACE_ID` must be defined only once in the value namespace of this module  
= note: this error originates in the attribute macro `contract`
```

Recommendation: We recommend reserving every generated `slots` item name before code generation, using the same normalization logic that produces field constant identifiers. At minimum, reject field identifiers whose generated slot constant is `NAMESPACE_ID` or `NAMESPACE_ROOT` when a contract-level namespace is present. A more robust fix is to build a set of all generated item identifiers for each field, including slot, offset, debug byte-size, collision-check, and namespace metadata names, and fail early with a clear `syn::Error` if any generated names collide.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. This is a compile-time issue only — the duplicate constant causes a hard build error rather than any runtime risk. The fix is to add `namespace_id` and `namespace_root` to the reserved field name list as a follow-up macro improvement.

3.2.13 Case-normalized field names can generate duplicate slot constants

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Summary: The precompile storage macros generate slot constant names by uppercasing field identifiers, but they do not reject field names that collide after this normalization. A storage layout containing case-distinct Rust fields such as `foo` and `F00` therefore reaches macro code generation and fails with duplicate generated constants instead of receiving a deterministic validation error.

Description: The macro layer accepts normal Rust field identifiers and later derives generated slot constant identifiers from a lossy uppercase representation of each field name. `PackingConstants::new` stores `const_name(name)`, and `const_name` uppercases the original identifier:

```
crates/common/precompile-macros/src/packing.rs#L16-L42 .
```

```
impl PackingConstants {  
    pub(crate) fn new(name: &Ident) -> Self {  
        Self(const_name(name))  
    }  
  
    pub(crate) fn slot(&self) -> Ident {  
        format_ident!("{}", &self.0)  
    }  
  
    pub(crate) fn offset(&self) -> Ident {  
        let span = proc_macro2::Span::call_site();  
        Ident::new(&format!("_{}_OFFSET", &self.0), span)  
    }  
}
```

```
pub(crate) fn const_name(name: &Ident) -> String {
    name.to_string().to_uppercase()
}
```

Those normalized names are then emitted as public slot and offset constants for every allocated field:

`crates/common/precompile-macros/src/packing.rs#L117-L153` .

```
for field in fields {
    let ty = field.ty;
    let consts = PackingConstants::new(field.name);
    let (loc_const, (slot_const, offset_const)) = (consts.location(),
    ↪ consts.into_tuple());
    // ...
    constants.extend(quote! {
        #[doc = #slot_doc]
        pub const #slot_const: ::alloy_primitives::U256 = #slot_expr;
        #[doc = #offset_doc]
        pub const #offset_const: usize = #offset_expr;
    });
}
```

The `#[contract]` parser only rejects exact reserved field names, so it does not catch case-normalized collisions before code generation:

`crates/common/precompile-macros/src/contract.rs#L98-L122` .

```
named_fields
    .into_iter()
    .map(|field| {
        let name = field
            .ident
            .as_ref()
            .ok_or_else(|| syn::Error::new_spanned(&field, "Fields must have names"))?;

        if RESERVED.contains(&name.to_string().as_str()) {
            return Err(syn::Error::new_spanned(
                name,
                format!("Field name '{name}' is reserved"),
            ));
        }
    })
}
```

The same packing path is also used by `#[derive(Storable)]` , which copies each original field identifier into `FieldInfo` and then calls `packing::allocate_slots` before generating the packing module:

`crates/common/precompile-macros/src/storable.rs#L75-L97` .

```
for field in fields {
    let name = field.ident.as_ref().unwrap().clone();
    field_infos.push(FieldInfo {
        name: name.clone(),
        ty: field.ty.clone(),
        slot: None,
        base_slot: None,
        namespace: None,
    });
    // ...
}

let layout_fields = packing::allocate_slots(&field_infos)?;
let packing_module = gen_packing_module_from_ir(&layout_fields, &mod_ident);
```

A minimal validation harness with a `#[contract]` storage struct containing both `foo` and `F00` reached macro expansion and failed with Rust duplicate-item errors for the generated constants `F00` , `F00_OFFSET` , and, in debug builds, `F00_BYTES` . This is not a Solidity restriction: Solidity identifiers are case-

sensitive, and an otherwise equivalent Solidity contract with `foo` and `F00` state variables compiles successfully. The mismatch matters because these Rust macros are intended to mirror Solidity-style storage layouts, but they impose an additional lossy normalization step on generated constant names. The currently inspected in-scope production storage layouts do not contain such a case-normalized duplicate pair, so the practical impact is limited to a future bad layout causing confusing generated-code compilation failures rather than current runtime storage corruption.

Proof of Concept: The following steps are self-contained except for requiring a checkout of `base/base` at the audited commit. They show that Solidity accepts case-distinct state variables while the Base Rust storage macro rejects the analogous layout only after generating duplicate constants.

First, check out the audited Base commit and install the Rust toolchain used by the workspace:

```
git clone https://github.com/base/base.git base-case-collision
cd base-case-collision
git checkout 6ee3da6325e7416812065c58358274a086f9f223
rustup toolchain install 1.94.1
```

Then compile an equivalent Solidity contract. `npm` downloads the pinned JavaScript `solc` package, so no global Solidity compiler install is required:

```
cat >/tmp/CaseCollision.sol <<'EOF'
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract CaseCollision {
    uint256 public foo;
    uint256 public F00;
}
EOF

npm --yes solc@0.8.26 --bin /tmp/CaseCollision.sol
```

This command succeeds, demonstrating that Solidity itself allows `foo` and `F00` as distinct identifiers.

Next, create a minimal Rust crate that depends on the local Base macro and storage crates from the checked-out repository:

```
BASE_REPO="$(pwd)"
POC_DIR="/tmp/base-case-slot-collision-poc"
rm -rf "$POC_DIR" /tmp/base-case-slot-collision-poc-target
mkdir -p "$POC_DIR/src"

cat >"$POC_DIR/Cargo.toml" <<EOF
[package]
name = "base-case-slot-collision-poc"
version = "0.1.0"
edition = "2024"

[workspace]

[dependencies]
revm = "38.0.0"
alloy-primitives = { version = "1.5.6", default-features = false }
base-precompile-macros = { path = "$BASE_REPO/crates/common/precompile-macros" }
base-precompile-storage = { path = "$BASE_REPO/crates/common/precompile-storage" }
EOF

cat >"$POC_DIR/src/lib.rs" <<'EOF'
use base_precompile_macros::contract;

#[contract]
pub struct CaseCollisionStorage {
    pub foo: u8,
    pub F00: u8,
```

```
}  
EOF
```

Finally, run the Rust compile check from outside the Base repository so Cargo does not load the repository-local linker configuration:

```
cd /tmp  
env -u RUSTFLAGS -u CARGO_ENCODED_RUSTFLAGS  
↪ CARGO_TARGET_DIR=/tmp/base-case-slot-collision-poc-target \  
cargo +1.94.1 check --manifest-path "$POC_DIR/Cargo.toml"
```

The Rust command fails with duplicate definitions for `F00`, `F00_OFFSET`, and `F00_BYTES`, all originating from the `#[contract]` attribute macro:

```
error[E0428]: the name `F00` is defined multiple times  
  = note: this error originates in the attribute macro `contract`  
  
error[E0428]: the name `F00_OFFSET` is defined multiple times  
  = note: this error originates in the attribute macro `contract`  
  
error[E0428]: the name `F00_BYTES` is defined multiple times  
  = note: this error originates in the attribute macro `contract`
```

Recommendation: We recommend tracking the normalized constant names during macro parsing and returning a `syn::Error` when two fields map to the same generated constant prefix. Apply the check to both `#[contract]` field parsing and `#[derive(Storable)]` struct parsing, and include both original field names plus the colliding generated prefix in the diagnostic. For example, reject `foo` and `F00` before emitting any `slots::F00`, `F00_OFFSET`, or packing-module constants.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. This is a compile-time issue only with no runtime impact and no current production layout affected. The root cause is shared with finding #38, and both will be addressed together in a follow-up macro improvement.

3.3 Informational

3.3.1 EIP-3541 comment documents the wrong `set_code` behavior

Severity: Informational.

Context: (No context files were provided by the reviewer).

Summary: `EvmPrecompileStorageProvider::set_code` contains a misleading EIP-3541 comment: the code only charges bytecode deposit gas and does not enforce EIP-3541's `0xef` first-byte rejection. The `0xef` marker itself is an intentional privileged precompile-factory marker and is not a protocol violation.

Description: Normal EVM contract creation rejects runtime bytecode whose first byte is `0xef` after London. The `revm` create finalization path performs that validation before depositing code:

```
// EIP-3541: Reject new contract code starting with the 0xEF byte  
if !is_eip3541_disabled  
    && spec_id.is_enabled_in(LONDON)  
    && interpreter_result.output.first() == Some(&0xEF)  
{  
    journal.checkpoint_revert(checkpoint);  
    interpreter_result.result = InstructionResult::CreateContractStartingWithEF;  
    return;  
}
```

The native precompile storage path does not use this creation finalization logic. `EvmPrecompileStorageProvider::set_code` charges code-deposit and account-creation gas, then directly delegates to `internals.set_code` at `crates/common/precompile-storage/src/evm.rs#L89-L123` :

```
fn set_code(&mut self, address: Address, code: Bytecode) -> Result<()> {
    let code_len = code.len();

    // EIP-3541 / Yellow Paper G_codedeposit: 200 gas per byte of deployed bytecode.
    self.deduct_gas(self.gas_params.code_deposit_cost(code_len))?;

    let is_new_account = { /* load account and check emptiness */ };

    if is_new_account {
        self.deduct_gas(self.gas_params.create_cost()?);
        /* hash and state gas charges */
    }

    self.internals
        .set_code(address, code)
        .map_err(|e| BasePrecompileError::Fatal(e.to_string()))
}
```

`alloy_evm::EvmInternals::set_code` only hashes and writes the bytecode to the account; it does not perform EIP-3541 first-byte validation. The audited code intentionally uses this privileged path to install `0xef` runtime bytecode in production flows. Generated precompile initialization writes a one-byte legacy `0xef` stub at `crates/common/precompile-macros/src/layout.rs#L183-L185` :

```
fn _initialize(&mut self) -> ::base_precompile_storage::Result<()> {
    let bytecode = ::revm::state::Bytecode::new_legacy(
        ::alloy_primitives::Bytes::from_static(&[0xef]),
    );
    self.storage.set_code(self.address, bytecode)?;
    Ok(())
}
```

The B20 factory uses the same pattern for token creation at `crates/common/precompiles/src/b20_factory/storage.rs#L69-L71` :

```
let checkpoint = self.storage.checkpoint();
let stub = Bytecode::new_legacy(Bytes::from_static(&[0xef]));
self.storage.set_code(token_address, stub)?;
```

Therefore, `CREATE` and `CREATE2` remain EIP-3541-compliant, while native precompile account creation/code installation can create code-bearing accounts whose runtime code starts with `0xef`. The customer confirmed that this distinction is intentional: because EIP-3541 prevents ordinary post-London `CREATE` / `CREATE2` deployments from producing runtime code beginning with `0xef`, a correctly validated `0xef`-prefixed native marker can serve as evidence that the account was created through the privileged precompile factory rather than as a normal contract.

The issue is limited to the nearby `set_code` comment. It refers to EIP-3541 while the line only charges Yellow Paper `G_codedeposit` gas and does not enforce the EIP-3541 first-byte rule. This makes the code appear to implement EIP-3541 behavior when it actually implements gas accounting for privileged code installation.

Impact Explanation: Impact is informational. The confirmed behavior does not create a direct funds-loss path or an EIP-3541 compliance failure for ordinary contract creation. The risk is reviewer and maintainer confusion: the comment documents an EIP-3541 enforcement behavior that the function does not provide, which can lead future changes to rely on a nonexistent validation check.

Likelihood Explanation: Likelihood is high because the misleading comment is present in the shared `set_code` path. Any reviewer or future maintainer reading the code can reasonably infer that the path handles EIP-3541 semantics, even though it only charges code-deposit gas.

Recommendation: Remove the EIP-3541 reference from the code-deposit gas comment, or replace it with wording that documents the intended split explicitly: ordinary post-London `CREATE / CREATE2` deployments reject `0xef` runtime code, while this privileged precompile storage path only charges code-deposit gas and may intentionally install a validated `0xef` marker.

Coinbase: Fixed in [PR 3230](#).

Cantina Managed: Fix verified.

3.3.2 Unused / unreachable code in the B20-asset and activation precompiles

Severity: Informational.

Context: [storage.rs#L143-L144](#), [dispatch.rs#L398-L400](#).

Description: Two spots contain code that is declared or executed but never effectively reached, leaving the implementation out of step with its stated intent.

1. Unreachable announcement reentrancy guard: In `b20_asset/dispatch.rs` the `announce` function rejects re-entry with `is_announcement_active()` :

```
if self.is_announcement_active() {  
    return Err(BasePrecompileError::revert(IB20Asset::AnnouncementInProgress {}));  
}
```

The `in_announcement` flag is only set after this check, by `begin_announcement()` in `token.rs` . The single re-entry vector is the internal call loop, which dispatches each call on the same `self` via `inner_with_privilege` while `in_announcement == true` . That loop already rejects any `announce` selector before dispatch:

```
if call_bytes[..4] == IB20Asset::announceCall::SELECTOR {  
    return Err(BasePrecompileError::revert(IB20Asset::AnnouncementInProgress {}));  
}
```

The only caller of `self.announce` is `SC::announce` . No other handler invokes it. So the selector check always fires first and the `is_announcement_active` branch can never be the control that catches a re-entrant call. The token is also built fresh per top level invocation with `in_announcement` set to false, so there is no cross invocation hole. The doc comment on `announce` overstates the flag's role. In practice the selector check does the work and `in_announcement` is a redundant backstop.

2. Unused AlreadyDeactivated ABI error: `AlreadyDeactivated(bytes32 feature)` is declared in `activation/abi.rs` but is never returned anywhere in the in scope crate. The only other matches in the tree are the unrelated `ValidatorAlreadyDeactivated` in the out of scope `.tempo-compare` reference. When `set_activated` deactivates a feature that is already off, it returns `FeatureNotActivated` instead, in `activation/storage.rs` :

```
let current = self.features.at(&feature).read()?;  
if current == activated {  
    if activated {  
        return Err(BasePrecompileError::revert(IActivationRegistry::AlreadyActivated {  
            feature,  
        }));  
    }  
  
    return Err(BasePrecompileError::revert(IActivationRegistry::FeatureNotActivated {  
        feature,  
    }));  
}
```

This reuses the same `FeatureNotActivated` error that `checkActivated`, which routes through `ensure_activated`, emits when querying a non activated feature. Callers cannot distinguish a deactivate of something already off from a feature that is not activated. The activate path has a symmetric error, `AlreadyActivated`, but the deactivate path has no in use counterpart, leaving `AlreadyDeactivated` dead.

Neither item is exploitable. Both are correctness hygiene issues where the code does not match its declared intent.

Recommendation:

1. Announcement guard: Keep the `is_announcement_active` check as cheap defense in depth, since it hardens against a future handler that might call `announce` indirectly, but correct the doc comment to state that the selector check is the active control and `in_announcement` is a redundant backstop. If minimalism is preferred, the check may be dropped in favor of the selector check alone, but retaining it costs nothing since it is an in memory bool.

2. AlreadyDeactivated error: Pick one consistent option.

First option, wire it up, which is preferred for symmetry. Return `IActivationRegistry::AlreadyDeactivated { feature }` in the already off branch of `set_activated`, mirroring the `AlreadyActivated` branch. This removes the collision with the `FeatureNotActivated` error used by `checkActivated` and makes the error set symmetric. Add or adjust a test asserting the deactivate when already off path reverts with `AlreadyDeactivated`.

Second option, drop it. Remove `AlreadyDeactivated` from the ABI to eliminate an unused, misleading declaration.

Coinbase: Fixed in [PR 3253](#).

Cantina Managed: Fix verified.

3.3.3 `StorageKey` rustdoc incorrectly documents divergent mapping key encodings as Solidity-compatible

Severity: Informational.

Context: [provider.rs#L315-L331](#).

Summary: The public `StorageKey::mapping_slot` rustdoc describes every supported mapping key as using Solidity mapping slot derivation, while the crate README documents intentional encoding divergences for signed integers and short fixed bytes.

Description: The precompile storage README explicitly documents that mapping slot derivation is only Solidity-compatible for a subset of key types. It states that unsigned integers, `Address`, `FixedBytes<32>`, and `String` match Solidity, but signed integers and `FixedBytes<N>` for `N < 32` intentionally diverge:

```
slot(key, base) = keccak256(lpad32(key) | to_be32(base))
```

[crates/common/precompile-storage/README.md#L53-L66](#).

The public rustdoc on `StorageKey::mapping_slot` omits those caveats and documents the method as a general Solidity mapping-slot implementation:

```
/// Computes `keccak256(lpad32(key) || slot_be32)` -- the Solidity mapping slot
↳ derivation.
fn mapping_slot(&self, slot: U256) -> U256 {
    let key_bytes = self.as_storage_bytes();
    let key_bytes = key_bytes.as_ref();
    debug_assert!(key_bytes.len() <= 32);
```

[crates/common/precompile-storage/src/provider.rs#L314-L330](#) .

The generated `StorageKey` implementations confirm that the README caveat is real. Signed Alloy integers use the raw signed value bytes, while `FixedBytes<N>` keys use `self.as_slice()` and are then left-padded by `mapping_slot` :

```
StorageKeyStrategy::SignedRaw(size) => quote! { self.into_raw().to_be_bytes::<#size>() },
StorageKeyStrategy::AsSlice => quote! { self.as_slice() },
```

[crates/common/precompile-macros/src/storable_primitives.rs#L56-L68](#) .

This creates inconsistent public guidance for integrators. A reader relying on the trait rustdoc can reasonably reconstruct off-chain mapping slots as if all `StorageKey` implementations matched Solidity `abi.encode(key, slot)` , while the README correctly warns that short fixed bytes and signed integer keys do not. The runtime behavior is internally consistent, but the rustdoc can lead external tooling, tests, or storage inspection scripts to derive the wrong keys for the divergent key types.

Recommendation: We recommend updating the `StorageKey::mapping_slot` rustdoc to mirror the README's compatibility caveat. The documentation should state that the formula is Solidity-compatible only for unsigned integers, `Address` , `FixedBytes<32>` , and string-keyed mappings, and that signed integers plus `FixedBytes<N>` for `N < 32` intentionally diverge. Consider linking directly to the `README` section so future changes keep the two public descriptions aligned.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.3.4 Packing module rustdoc uses stale `Layout::Slot` and misstates fixed-array layout behavior

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Summary: The `precompile-storage` packing module rustdoc references a non-existent `Layout::Slot` variant and incorrectly states that fixed-size arrays are non-primitives using that layout, while the generated implementations use `Layout::Slots(...)` and pack eligible primitive array elements.

Description: The module-level rustdoc for the packing utilities describes the packing model as follows:

```
//! Packing only applies to primitive types where `LAYOUT::Bytes(count) && count < 32`.
//! Non-primitives (structs, fixed-size arrays, dynamic types) have `LAYOUT =
↳ Layout::Slot`.
```

[crates/common/precompile-storage/src/packing.rs#L1-L13](#) .

This documentation is stale. The storage layout enum does not define `Layout::Slot` ; it defines byte-sized layouts and multi-slot layouts. Array and struct-like values use `Layout::Slots(...)` , not `Layout::Slot` .

```
pub enum Layout {
    /// Single slot, N bytes (1-32). Can be packed with other fields if N < 32.
    Bytes(usize),
    /// Occupies N full slots. Cannot be packed.
    Slots(usize),
}
```

[crates/common/precompile-storage/src/provider.rs#L173-L180](#) .

The generated fixed-size array implementation also contradicts the rustdoc. Arrays are emitted with `Layout::Slots(slot_count)` , where `slot_count` is calculated from packed element capacity when the element type is smaller than 32 bytes:

```

let slot_count_expr = if *elem_is_packable {
    quote! { ::base_precompile_storage::calc_packed_slot_count(#array_size,
        ↪ #elem_byte_count) }
} else {
    quote! { #array_size }
};

impl ::base_precompile_storage::StorableType for [#elem_type; #array_size] {
    const LAYOUT: ::base_precompile_storage::Layout =
        ↪ ::base_precompile_storage::Layout::Slots(#slot_count_expr);
}

```

[crates/common/precompile-macros/src/storable_primitives.rs#L293-L317](#).

The generated array load/store paths then use `calc_element_slot`, `calc_element_offset`, `extract_from_word`, and `insert_into_word` for packable primitive elements:

```

let slot_idx = calc_element_slot(i, #elem_byte_count);
let offset = calc_element_offset(i, #elem_byte_count);
let slot_addr = base_slot + ::alloy_primitives::U256::from(slot_idx);
let slot_value = storage.load(slot_addr)?;
result[i] = extract_from_word(slot_value, offset, #elem_byte_count)?;

```

```

let slot_count = ::base_precompile_storage::calc_packed_slot_count(#array_size,
    ↪ #elem_byte_count);
for slot_idx in 0..slot_count {
    let slot_addr = base_slot + ::alloy_primitives::U256::from(slot_idx);
    let mut slot_value = ::alloy_primitives::U256::ZERO;
}

```

[crates/common/precompile-macros/src/storable_primitives.rs#L350-L379](#).

The dedicated array handler documentation states the current behavior correctly: fixed-size arrays start at their base slot, and small elements are packed while larger elements use full slots.

```

/// Fixed-size arrays `[T; N]` use Solidity-compatible array storage:
/// - **Base slot**: Arrays start directly at `base_slot` (not at keccak256)
/// - Small elements (`T::BYTES` <= 16) are packed; larger elements use full slots.

```

[crates/common/precompile-storage/src/types/array.rs#L1-L5](#).

The runtime behavior is not broken by this rustdoc, but the public module documentation gives integrators and reviewers the wrong mental model for fixed-size array storage. This can cause storage-layout reviews, migration notes, or off-chain decoding tools to reserve one slot per fixed-array element even when the generated implementation packs eligible elements into fewer slots.

Recommendation: We recommend updating the `packing.rs` module rustdoc to use the real enum variant names and describe fixed-size arrays separately. The documentation should state that primitives use `Layout::Bytes(count)`, multi-slot values use `Layout::Slots(count)`, and fixed-size arrays use `Layout::Slots(...)` with primitive elements packed according to `calc_packed_slot_count` when eligible.

Coinbase: Fixed in PR 3280.

Cantina Managed: Fix verified.

3.3.5 Generated store logic carries stale comments referencing nonexistent upstream code

Severity: Informational.

Context: [storable.rs#L661](#), [storable.rs#L688](#).

Description: The store-implementation codegen emits the same comment in two places. Both comments reference an `is_t4()` helper and a "Category N" taxonomy. Neither `is_t4()` nor the "Category N"

classification exists anywhere in this repository, as confirmed by search. They are leftovers carried over from the upstream Tempo precompiles-macros.

The practical effect is that an auditor or reviewer reading the generated store logic is given a false mental model, implying that a specific optimization once existed here and was deliberately disabled. No such optimization is present in this codebase, so the comments are misleading. This is the same category of issue as the other misleading-documentation findings.

Recommendation: Remove or rewrite both comments so they describe the actual behavior of the current code. The intended meaning is simply that the slot is always loaded before packing, so the comment should state that directly without referencing the removed `is_t4()` helper or the "Category N" taxonomy.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.3.6 Mapping-Only Storable Structs Emit Invalid `IS_DYNAMIC`

Severity: Informational.

Context: [storable.rs#L131-L132](#).

Description: A struct deriving `#[derive(Storable)]` with only mapping fields produces uncompileable code. The macro builds `IS_DYNAMIC` by joining the direct field types with `||` via a `quote!` repetition. When every field is a mapping, `direct_tys` is empty, the repetition expands to nothing, and the result is `const IS_DYNAMIC: bool = ;`, which rustc rejects with an "expected expression" error.

Recommendation: Emit `const IS_DYNAMIC: bool = false;` when `direct_tys` is empty.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.3.7 Decimals Default Uses a Magic Literal Instead of the Named Bound

Severity: Informational.

Context: [accounting.rs#L373-L375](#).

Description: The generated `decimals` accessor falls back to a hardcoded literal `6` when the stored value is zero. The canonical source of this default is `B20AssetStorage::MIN_DECIMALS`:

```
pub const MIN_DECIMALS: u8 = 6;
```

The standalone `B20AssetStorage::decimals` method in the same file correctly uses `Self::MIN_DECIMALS` for its fallback:

```
pub fn decimals(&self) -> Result<u8> {
    let decimals = self.asset.decimals()?;
    Ok(if decimals == 0 { Self::MIN_DECIMALS } else { decimals })
}
```

There are now two unsynchronized definitions of the same default. They agree today because both are `6`, but if `MIN_DECIMALS` were ever changed the generated macro path would silently diverge from the hand-written accessor while neither produces a compile error.

This is an internal inconsistency rather than a behavioral bug. The `multiplier` fallback immediately above it in the same file does this correctly by referring to the named `Self::WAD` constant instead of an inline literal, so the hardcoded `6` stands out as the odd case.

Recommendation: Replace the literal `6` with the named constant `B20AssetStorage::MIN_DECIMALS` so the generated accessor and the hand-written accessor share a single source of truth, matching how the `multiplier` fallback already uses `Self::WAD`.

Coinbase: Fixed in PR 3307.

Cantina Managed: Fix verified.

3.3.8 Dead Code in Generated `role_admin` Default Fallback Branch

Severity: Informational.

Context: [accounting.rs#L218-L226](#).

Description: The generated `role_admin` accessor is intended to express the convention that an unset admin role defaults to `DEFAULT_ADMIN_ROLE`. In practice the conditional is a no-op. `B20TokenRole::DefaultAdmin.id()` resolves to `B256::ZERO`:

```
pub const fn id(self) -> B256 {
    match self {
        Self::DefaultAdmin => B256::ZERO,
```

In the `if` branch the guard `admin_role.is_zero()` is true and the returned value is `DefaultAdmin.id()`, which equals `B256::ZERO`, the exact value `admin_role` already holds. Both arms therefore return identical values and the conditional has no observable effect.

The behavior is correct today only because the OpenZeppelin convention happens to make `DEFAULT_ADMIN_ROLE` equal to zero. The intended "unset admin resolves to the default admin" guarantee is not actually implemented in this code. It is accidentally satisfied by the zero constant. If `DefaultAdmin.id()` were ever changed to a nonzero value, this fallback would silently break. An unset role's admin would resolve to the zero role rather than the default admin, so an `ensure_role` check against `role_admin` would validate against the wrong role. Notably, the upstream Tempo equivalent has no such branch. It reads the stored admin directly and documents that an unset entry reads as zero, which is the default admin. The branch addition here gives a false impression of implementing a fallback.

Recommendation: Either remove the conditional and read the stored admin directly, matching the upstream model where an unset entry is zero and zero is the default admin, or implement the fallback so it does not depend on the default admin being zero. If the fallback is retained for clarity, add a comment stating that it relies on `DEFAULT_ADMIN_ROLE` being zero so the dependency is explicit rather than implicit.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. The conditional has no observable effect today because `DEFAULT_ADMIN_ROLE` equals `B256::ZERO`, making both arms of the branch return the same value. Coinbase will proceed with removing the redundant conditional.

3.3.9 Namespace macro contract branch is unreachable under documented usage

Severity: Informational.

Context: [namespace.rs#L31-L35](#).

Description: Attribute macros expand outermost first. In the documented layout, `#[contract]` is written above `#[namespace()]`, so the compiler expands `#[contract]` first and that macro receives the struct with the `#[namespace]` attribute still attached. `#[contract]` reads the namespace directly through `extract_namespace`, emits the final struct, and `namespace::expand` never runs. The contract branch in `namespace.rs` therefore never executes for the documented ordering.

That branch only runs when the attributes are inverted, with `#[namespace()]` written above `#[contract]`. This ordering is not documented and appears in exactly one place, the `namespace_outer_`

`order` test, which keeps the branch alive. Both orderings produce the same correct output, so this is a maintainability issue rather than a behavioral bug.

The same dead path also contains redundant validation. The early `parse_namespace_id` call in `expand_impl` validates the namespace id purely for its side effect and discards the result, then forwards the raw literal downstream where `extract_namespace` and `extract_storage_namespace` parse and validate it again. The two validation sites could drift if one is changed without the other.

Recommendation: Remove the contract branch and the inverted order test, and let `#[contract]` own namespace handling exclusively, or document and support the attribute ordering as an explicit contract. In either case drop the redundant `parse_namespace_id` call so namespace validation has a single source of truth.

Coinbase: Fixed in [PR 3308](#).

Cantina Managed: Fix verified.

3.3.10 `install` Parser Reports a Misleading Error for Unsupported Options

Severity: Informational.

Context: [precompile.rs#L191-L195](#).

Description: `InstallConfig::parse` handles the contents of `install()` for the `#[precompile]` macro. It parses `address = <expr>` or `addr = <expr>`, then tries to enforce that nothing else follows. The intent is that an unsupported trailing token produces the dedicated message `unexpected install option`, which names the real problem for the macro author.

The two trailing checks run in the wrong order. After the address expression is parsed, the first block tests whether any input remains and, if so, immediately requires a comma. For the common malformed case `install(address = X extra)`, the leftover token is `extra`, not a comma, so `parse::<Token![,]>()` fails first and emits an expected comma error pointing at `extra`. The author is told a comma is missing when the actual issue is that `extra` is an unsupported option. The dedicated `unexpected install option` message on the second block is only reachable when a comma was genuinely present, as in `install(address = X, extra)`, so the clearer diagnostic never fires for the no comma case.

Every valid invocation, `install`, `install(address = X)`, and `install(addr = X)`, parses correctly, and a single trailing comma is tolerated. The defect only affects the compile error text shown for malformed input, where the macro misattributes the cause.

Recommendation: Check for leftover tokens before requiring the comma, so the parser reports the unsupported option directly rather than complaining about a missing separator. Peek for a comma first, treat any other remaining token as an unexpected option, then consume the optional comma and reconfirm the input is empty:

```
if !input.is_empty() && !input.peek(Token![,]) {
    return Err(syn::Error::new(input.span(), "unexpected `install` option"));
}
if !input.is_empty() {
    input.parse::<Token![,]>()?;
}
if !input.is_empty() {
    return Err(syn::Error::new(input.span(), "unexpected `install` option"));
}
```

This makes the `unexpected install option` message the one the author sees for `install(address = X extra)` while preserving the existing behavior for valid input and a single trailing comma.

Coinbase: Fixed in [PR 3309](#).

Cantina Managed: Fix verified.

3.3.11 `with_caller` uses a redundant manual `drop` and restores the caller through a panicking borrow

Severity: Informational.

Context: `storage_ctx.rs#L179`.

Description: Restoration of the previous caller is already guaranteed by `CallerGuard`'s `Drop`, which runs at end of scope. The explicit `let result = f(); drop(guard); result` therefore adds nothing. Having `let _guard = ; f()` is equivalent, and only obscures that RAI, not the manual `drop`, is what restores the caller.

Separately, `CallerGuard::drop` restores through `with_storage`, which calls `RefCell::borrow_mut()` and panics on a conflicting borrow, instead of the fallible `try_with_storage` used elsewhere. If `with_caller` were called while the storage cell is already borrowed, the restore could panic inside `Drop` during unwinding and abort the process.

Recommendation: Primary fix: keep `CallerGuard`, but remove the manual `drop(guard)` and bind it as `_guard` so the guard's existing `Drop` restores the caller on scope exit:

```
pub fn with_caller<R>(&self, caller: Address, f: impl FnOnce() -> R) -> R {
    let previous = self.with_storage(|s| s.replace_caller(caller));
    let _guard = CallerGuard { storage: *self, previous: Some(previous) };
    f()
}
```

Optional hardening: replace the body of the existing `CallerGuard::drop` so it restores through `try_borrow_mut`. A conflicting borrow then skips the restore instead of panicking inside `Drop` (which would abort during unwinding):

```
impl Drop for CallerGuard<'_> {
    fn drop(&mut self) {
        if let Some(previous) = self.previous.take() {
            if let Ok(mut guard) = self.storage.storage.try_borrow_mut() {
                guard.replace_caller(previous);
            }
        }
    }
}
```

Coinbase: Fixed in PR 3404.

Cantina Managed: Fix verified.

3.3.12 Contract macro diagnostic omits supported address attribute

Severity: Informational.

Context: `contract.rs#L27`.

Summary: The `#[contract]` macro accepts both `addr = ...` and `address = ...`, but its unsupported-attribute diagnostic says only `addr` is supported.

Description: The `#[contract]` attribute macro parses its attribute stream through `ContractConfig` in `crates/common/precompile-macros`. The parser accepts an empty attribute list, `addr = ...`, or `address = ...`; any other identifier is rejected.

`contract.rs`.

```
impl syn::parse::Parse for ContractConfig {
    fn parse(input: ParseStream<'_>) -> syn::Result<Self> {
```

```

    if input.is_empty() {
        return Ok(Self { address: None });
    }

    let ident: Ident = input.parse()?;
    if ident != "addr" && ident != "address" {
        return Err(syn::Error::new(ident.span(), "only `addr` attribute is
        ↪ supported"));
    }

    input.parse::()?;
    let address: Expr = input.parse()?;

    Ok(Self { address: Some(address) })
}

```

The rejection message is therefore stale: the actual accepted key set includes both `addr` and `address`, while the diagnostic says only `addr` is supported. This does not affect generated runtime behavior, but it gives incorrect guidance to developers working with the audited precompile macro. A caller who misspells another key, or who is trying to determine the supported public macro syntax from compiler output, is told that `address` is unsupported even though the parser intentionally accepts it.

The public macro entrypoint forwards the parsed `ContractConfig` directly into contract generation, so the accepted `address = ...` alias follows the same generation path as `addr = ...`.

`lib.rs`.

```

#[proc_macro_attribute]
pub fn contract(attr: TokenStream, item: TokenStream) -> TokenStream {
    let config = parse_macro_input!(attr as contract::ContractConfig);
    let input = parse_macro_input!(item as DeriveInput);
    contract::generate(input, config.address.as_ref())
}

```

Recommendation: We recommend updating the diagnostic to describe the full accepted key set, for example:

```

return Err(syn::Error::new(
    ident.span(),
    "only `addr` or `address` attribute is supported",
));

```

Coinbase: Fixed in [PR 3388](#).

Cantina Managed: Fix verified.

3.3.13 Precompile install diagnostic omits supported `addr` attribute

Severity: Informational.

Context: [precompile.rs#L185](#).

Summary: The `#[precompile]` macro accepts both `install(address = ...)` and `install(addr = ...)`, but its unsupported-key diagnostic says only `address = ...` is supported.

Description: The `#[precompile]` install-option parser supports two equivalent keys for selecting the installed precompile address. The parser rejects only keys that are neither `address` nor `addr`, so both spellings are part of the public macro syntax:

```

impl Parse for InstallConfig {
    fn parse(input: ParseStream<'_>) -> syn::Result<Self> {

```

```

let key: Ident = input.parse()?;
if key != "address" && key != "addr" {
    return Err(syn::Error::new_spanned(key, "`install` supports only `address =
    ↪ ...`"));
}

```

`precompile.rs` .

The diagnostic is stale because it documents only the long-form key. A precompile author who misspells an install option, or who relies on compiler output to determine the accepted syntax, is told that only `address = ...` is supported even though `addr = ...` is accepted by the same parser.

The parsed install address is then used by the generated `install()` method, confirming that the accepted `addr` alias reaches the user-facing macro generation path rather than being dead parsing logic:

```

let install = config.install.map(|install| {
    let address = install
        .address
        .map_or_else(|| quote! { <#storage>::ADDRESS }, |address| quote! { #address });

    quote! {
        pub fn install(
            precompiles: &mut ::alloy_evm::precompiles::PrecompilesMap,
            #(#install_arg_defs),*
        ) {
            precompiles.extend_precompiles(::core::iter::once((
                #address,
                Self::precompile(#(#install_arg_names),*),
            )));
        }
    }
});

```

`precompile.rs` .

Recommendation: We recommend updating the diagnostic so it documents the full accepted key set, for example:

```

return Err(syn::Error::new_spanned(
    key,
    "`install` supports only `address = ...` or `addr = ...`",
));

```

Coinbase: Fixed in [PR 3383](#).

Cantina Managed: Fix verified.

3.3.14 Stablecoin decimals Fallback Returns 0 Instead of the System Default

Severity: Informational.

Context: [accounting.rs#L26-L30](#).

Description: This non-asset `decimals` branch, taken only by `B20StablecoinStorage`, maps a `None` from `B20Variant::from_address` to `0`, the exact sentinel every other decimals path treats as "unset, fall back to `6`". The `0` is unreachable today because the stablecoin storage is only ever constructed from an address already validated as a stablecoin B-20 address (by `lookup.rs` and by the factory's `compute_address`), so `from_address` always returns `Some(Stablecoin)` and `decimals()` always yields `6`. This is therefore not a behavioral bug, but the fallback relies on an invariant enforced two layers away; if the address were ever arbitrary the `0` would silently leak instead of falling back to the documented default, and a token reporting `decimals == 0` is anomalous for an ERC-20-style asset. This is the sibling of Finding 4, which flags the asset path hardcoding `6` where this one hardcodes `0`.

Recommendation: Replace the `0` fallback with the canonical default so it matches the rest of the system and no longer depends on an externally enforced invariant. For example, fall back to `crate::B20Variant::Stablecoin.decimals()` so the unreachable arm still produces the documented `6` rather than the “unset” sentinel.

Coinbase: Fixed in [PR 3385](#).

Cantina Managed: Fix verified.

3.3.15 `#[slot(key)]` computes a non ERC 7201 slot and misleads on namespacing

Severity: Informational.

Context: [utils.rs#L34-L37](#).

Description: `#[slot("key")]` and `#[base_slot("key")]` compute a plain `keccak256("key")`. This is not ERC 7201. It skips the minus 1 step, the second hash, and the final `& ~0xff` mask. The mask is what matters: ERC 7201 zeroes the low byte so the 256 slots following a root cannot overlap another namespace root. The string form drops that guarantee.

The real risk is semantic, not a runtime hash collision. The attribute name implies safe namespacing that it does not provide, so `#[slot("mylabel")]` quietly lands on a raw unmasked keccak slot with no structural protection against layout overlap. Base already has the correct mechanism in `#[namespace()]`, backed by `erc7201_root`.

`precompile-storage/README.md` makes it worse by listing `#[slot("key")]` under “Manual slot override” next to the integer forms, presenting it as an equal blessed option. No Base contract uses the string form, so this is unused but documented surface.

Recommendation: Preferred: delete the `Lit::Str` arm and the `#[slot("key")]` README line so `#[slot]` and `#[base_slot]` take integers only. All hashed namespacing then goes through `#[namespace]`. No in tree usage means this is non breaking for Base.

Fallback for Tempo parity: keep the arm but document the string form in code and README as a raw non ERC 7201 primitive with no namespace separation, and move the README line out of the override list so it is not presented as equivalent.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.3.16 Release builds do not enforce generated storage collision checks

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Summary: The generated storage constructor compiles out the only collision-enforcement hook in release builds.

Description: The generated collision checks are all gated behind `debug_assertions`:

- The generated `__new` constructor calls `slots::__check_all_collisions()` only when `debug_assertions` are enabled at `layout.rs`.
- The aggregate collision function is emitted only under `#[cfg(debug_assertions)]` at `layout.rs`.
- Each per-field check is emitted only under `#[cfg(debug_assertions)]` at `packing.rs`.

The constructor then unconditionally instantiates field handlers from generated slot constants. Release builds therefore trust the generated layout even if overlapping field locations exist.

Relevant code:

```
fn __new(address: ::alloy_primitives::Address, storage:
↳ ::base_precompile_storage::StorageCtx<'a>) -> Self {
    #[cfg(debug_assertions)]
    {
        slots::__check_all_collisions();
    }

    Self { /* field handlers */ address, storage }
}

#[cfg(debug_assertions)]
pub(super) fn __check_all_collisions() {
    #(#check_fn_calls();)*
}
```

Recommendation: We recommend removing the build-profile gate around generated collision checks or converting collision detection into a compile-time macro error before code generation completes.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.3.17 `args()` empty-list duplicate is not rejected

Severity: Informational.

Context: `precompile.rs#L116`.

Description: Every other option (`id`, `storage`, `macro_path`, `install`) rejects duplicates via `reject_duplicate(&..., &key)`, which keys off "has the key been seen." `args` instead guards on `!args.is_empty()`, which keys off the parsed value. Because an empty `args()` leaves the vec empty, the guard does not fire:

- `args()`, `args(x: u8)` is accepted; the second silently overwrites the first.
- `args()`, `args()` is accepted.

A duplicate is only caught when the first `args()` is non-empty, making detection inconsistent with the rest of the parser. This is a malformed-input lint gap a developer hits by hand-writing redundant macro input; generated code remains correct for whatever final `args` value wins.

Recommendation: Track presence rather than emptiness, matching the `reject_duplicate` pattern. Change `args` to `Option<Vec<PrecompileArg>>`:

```
"args" => {
    reject_duplicate(&args, &key)?;
    let content;
    parenthesized!(content in input);
    args = Some(
        content
            .parse_terminated(PrecompileArg::parse, Token![,])?
            .into_iter()
            .collect(),
    );
}
```

Coinbase: Fixed in [PR 3405](#).

Cantina Managed: Fix verified.

3.3.18 `#[contract]` silently drops all non-`derive` struct-level attributes

Severity: Informational.

Context: `contract.rs#L64-L69`.

Description: `gen_output` keeps only `#[derive()]` and filters out every other struct-level outer attribute. Doc comments, `#[cfg]`, `#[cfg_attr]`, `#[allow]`, `#[deprecated]`, and `#[serde]` never reach the regenerated `struct #name<'a>`. The `derives` vec is the only attribute set threaded into `gen_struct`, which also hardcodes its own `#[doc]` at `layout.rs:129-137`, so any user doc comment is replaced by the generic "Storage layout for the [Name] precompile." string.

This is live. `PolicyRegistryStorage` at `storage.rs:59-64` carries a hardfork-safety note that the macro discards. The doc `Slots` are append-only, never reorder across hardforks never reaches the generated type's rustdoc.

The whitelist direction is wrong. Field-level attributes must be dropped because the generated struct has different fields, and the macro helper attributes `contract`, `namespace`, and `storage_namespace` must be stripped. Ordinary struct-level outer attributes belong on the regenerated type, but whitelisting `derive` drops them too.

Recommendation: Forward all struct-level attributes except the macro helpers, and stop clobbering user docs. Replace the `derive` whitelist with a blacklist of macro helper attributes using the existing `attr_path_is` helper at `utils.rs:43`.

```
let forwarded = input
    .attrs
    .iter()
    .filter(|attr| {
        !attr.path().is_ident("contract")
        && !attr_path_is(attr.path(), "namespace")
        && !attr_path_is(attr.path(), "storage_namespace")
    })
    .cloned()
    .collect::<Vec<_>>();
```

Then in `gen_struct` at `layout.rs:129`, emit the fallback `#[doc = #doc_str]` only when the forwarded attrs contain no `doc` attribute, so a user doc comment is preserved instead of duplicated. Then in `gen_struct` at `layout.rs:129`, emit the fallback `#[doc = #doc_str]` only when the forwarded attrs contain no `doc` attribute, so a user doc comment is preserved instead of duplicated.

Coinbase: Fixed in [PR 3407](#).

Cantina Managed: Fix verified.

3.3.19 `STATICCALL` state-change violations are returned as ordinary reverts

Severity: Informational.

Context: (No context files were provided by the reviewer).

Summary: Base native precompile storage rejects `sstore`, `tstore`, and event emission while executing in a static context, but the shared error conversion maps those violations to an ordinary `Precompile Output::revert` with empty returndata. This makes static-context state-change attempts from native precompiles use revm's normal revert path instead of the exceptional static-call state-change halt used by EVM opcodes.

Description: `EvmPrecompileStorageProvider` correctly detects static execution before mutating persistent storage, transient storage, or logs and returns `BasePrecompileError::StaticCallViolation`:

```

fn sstore(&mut self, address: Address, key: U256, value: U256) -> Result<()> {
    if self.is_static {
        return Err(BasePrecompileError::StaticCallViolation);
    }
    // ...
}

fn tstore(&mut self, address: Address, key: U256, value: U256) -> Result<()> {
    if self.is_static {
        return Err(BasePrecompileError::StaticCallViolation);
    }
    // ...
}

fn emit_event(&mut self, address: Address, event: LogData) -> Result<()> {
    if self.is_static {
        return Err(BasePrecompileError::StaticCallViolation);
    }
    // ...
}

```

evm.rs .

However, `BasePrecompileError::into_precompile_result` classifies `StaticCallViolation` as empty revert data and then returns `PrecompileOutput::revert(...)` :

```

pub fn into_precompile_result(self, gas: u64, state_gas: u64) -> PrecompileResult {
    let bytes: Bytes = match self {
        // ...
        Self::StaticCallViolation => Bytes::new(),
        // ...
    };
    Ok(PrecompileOutput::revert(gas, bytes, state_gas))
}

```

error.rs .

In the pinned revm execution path, a reverted precompile output is converted to `InstructionResult::Revert`, while a halted precompile output is converted to a precompile halt result. Opcode-level static-call state-change checks use the halt path instead: revm's static-call guard returns `InstructionResult::StateChangeDuringStaticCall`, which converts to `HaltReason::StateChangeDuringStaticCall`.

The broken guarantee is therefore not that the static guard is missing. The guard exists, but it is lowered into the same result class as contract-defined reverts. Static state-change attempts against native precompiles can be observed as an ordinary empty revert instead of the EVM's static-call state-change exceptional halt, including different result classification and gas/returndata handling from the opcode path.

Recommendation: We recommend mapping `BasePrecompileError::StaticCallViolation` to a halt/error result instead of `PrecompileOutput::revert`. The fix should make static-context mutations from native precompiles follow the same exceptional path as opcode-level `StateChangeDuringStaticCall`, and tests should assert the final precompile/interpreter result class rather than only checking that the storage provider returned `StaticCallViolation`.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. The static-call guard is correct and enforced — `sstore`, `tstore`, and event emission all check `is_static` and return an error before any state mutation occurs. The issue is purely one of error classification: `StaticCallViolation` is lowered to `PrecompileOutput::revert` rather than a halt, producing `InstructionResult::Revert` instead of `HaltReason::StateChangeDuringStaticCall`. No state change is possible. Improving the classification for EVM spec conformance is tracked as a follow-up.

4 Appendix

4.1 Unit Test Coverage Improvements

This section documents the unit tests added to improve coverage of the `base-precompile-storage` and `base-precompile-macros` crates. Each subsection lists the source file, a table of the new tests with what they verify and their pass status, followed by the test source code.

Result: `base-precompile-macros` 48 passed / 0 failed, `base-precompile-storage` 148 passed / 0 failed. Every test listed below passed.

Crate	File	New tests
<code>base-precompile-storage</code>	<code>src/evm.rs</code>	5
<code>base-precompile-storage</code>	<code>src/provider.rs</code>	6
<code>base-precompile-macros</code>	<code>src/utils.rs</code>	13
<code>base-precompile-macros</code>	<code>src/accounting.rs</code>	5
<code>base-precompile-macros</code>	<code>src/storable.rs</code>	1
<code>base-precompile-macros</code>	<code>src/contract.rs</code>	9
<code>base-precompile-macros</code>	<code>src/namespace.rs</code>	6
Total	7 files	45

4.1.1 `crates/common/precompile-storage/src/evm.rs`

These tests exercise the production `EvmPrecompileStorageProvider` against a real revm journal. Prior coverage only exercised the `HashMapStorageProvider` mirror.

Test	What is tested	Pass
<code>state_gas_is_drawn_from_regular_gas_on_new_account</code>	State gas is drawn from regular gas; <code>gas_used()</code> includes both non-state and state-gas components	✓
<code>reservoir_is_zero_even_with_nonzero_input_reservoir</code>	<code>reservoir()</code> reports 0 regardless of input reservoir value	✓
<code>set_code_fails_closed_when_gas_excludes_state_gas</code>	<code>set_code</code> returns <code>OutOfGas</code> when the budget covers only non-state costs	✓
<code>sstore_in_static_context_is_rejected</code>	<code>sstore</code> in a static call returns <code>StaticCall Violation</code>	✓
<code>keccak256_hashes_and_charges_gas</code>	<code>keccak256</code> returns the correct hash and charges base + per-word gas	✓

```
/// On the production provider, the partial-EIP-8037 path draws state gas from
/// regular gas: a new-account `set_code` counts the state gas in BOTH
/// `state_gas_used()` and the regular `gas_used()` total.
#[test]
fn state_gas_is_drawn_from_regular_gas_on_new_account() {
    let mut ctx = ctx();
    let gas_params = gas_params();
    let code = code();
    let len = code.len();

    let mut provider =
        EvmPrecompileStorageProvider::new(make_input(&mut ctx, u64::MAX, 0, false),
            ↪ gas_params.clone());
    provider.set_code(ADDR, code).unwrap();

    let state_gas = gas_params.create_state_gas() +
        ↪ gas_params.code_deposit_state_gas(len);
```

```

    assert!(state_gas > 0, "AMSTERDAM state gas must be non-zero");
    assert_eq!(provider.state_gas_used(), state_gas);
    assert_eq!(provider.gas_used(), non_state_regular(&gas_params, len) + state_gas);
    assert!(provider.gas_used() > provider.state_gas_used());
}

/// The provider reports pre-fork reservoir semantics: `reservoir()` is always 0,
/// even with a non-zero input reservoir and after state gas is charged.
#[test]
fn reservoir_is_zero_even_with_nonzero_input_reservoir() {
    let mut ctx = ctx();
    let mut provider =
        EvmPrecompileStorageProvider::new(make_input(&mut ctx, u64::MAX, 5_000, false),
            ↪ gas_params());

    assert_eq!(provider.reservoir(), 0, "input reservoir must be ignored (pre-fork =
    ↪ 0)");
    provider.set_code(ADDR, code()).unwrap();
    assert_eq!(provider.reservoir(), 0, "charging state gas must not create a
    ↪ reservoir");
}

/// Because state gas is drawn from regular gas, a budget that only covers the
/// non-state costs makes `set_code` fail closed on the state-gas charge.
#[test]
fn set_code_fails_closed_when_gas_excludes_state_gas() {
    let mut ctx = ctx();
    let gas_params = gas_params();
    let code = code();
    let budget = non_state_regular(&gas_params, code.len());

    let mut provider =
        EvmPrecompileStorageProvider::new(make_input(&mut ctx, budget, 0, false),
            ↪ gas_params());
    let result = provider.set_code(ADDR, code);

    assert!(matches!(result, Err(BasePrecompileError::OutOfGas)));
}

/// `sstore` in a static call context must revert with `StaticCallViolation`
/// before touching the journal.
#[test]
fn sstore_in_static_context_is_rejected() {
    let mut ctx = ctx();
    let mut provider =
        EvmPrecompileStorageProvider::new(make_input(&mut ctx, u64::MAX, 0, true),
            ↪ gas_params());

    let result = provider.sstore(ADDR, U256::from(1), U256::from(2));
    assert!(matches!(result, Err(BasePrecompileError::StaticCallViolation)));
}

/// `keccak256` returns the correct hash and charges base + per-word gas.
#[test]
fn keccak256_hashes_and_charges_gas() {
    let mut ctx = ctx();
    let mut provider =
        EvmPrecompileStorageProvider::new(make_input(&mut ctx, u64::MAX, 0, false),
            ↪ gas_params());

    let data = b"base precompile keccak input spanning two words!!";
    let before = provider.gas_used();
    let hash = provider.keccak256(data).unwrap();

    assert_eq!(hash, keccak256(data));
    let num_words = data.len().div_ceil(32) as u64;

```

```
    assert_eq!(provider.gas_used() - before, KECCAK256 + KECCAK256WORD * num_words);
}
```

4.1.2 crates/common/precompile-storage/src/provider.rs

A new test module covering the storage-trait core logic (the file previously had no tests).

Test	What is tested	Pass
layout_is_packable	Layout::Bytes / Layout::Slots packability boundaries	✓
layout_slots_and_bytes	Slot and byte count math for layout types	✓
layout_ctx_packed_offset	LayoutCtx::FULL vs packed(n) offset behavior	✓
mapping_slot_matches_solidity_derivation	StorageKey::mapping_slot matches Solidity keccak256(lpad32(key) slot)	✓
packable_full_roundtrip_and_delete	Blanket Storable for Packable : full-slot store/load/delete	✓
packable_packed_delete_preserves_neighbor_byte	Packed delete clears only its field; an adjacent byte is preserved	✓

```
#[test]
fn layout_is_packable() {
    assert!(Layout::Bytes(1).is_packable());
    assert!(Layout::Bytes(31).is_packable());
    assert!(!Layout::Bytes(32).is_packable());
    assert!(!Layout::Slots(1).is_packable());
}

#[test]
fn layout_slots_and_bytes() {
    assert_eq!(Layout::Bytes(20).slots(), 1);
    assert_eq!(Layout::Bytes(20).bytes(), 20);
    assert_eq!(Layout::Slots(3).slots(), 3);
    assert_eq!(Layout::Slots(3).bytes(), 96);
}

#[test]
fn layout_ctx_packed_offset() {
    assert_eq!(LayoutCtx::FULL.packed_offset(), None);
    assert_eq!(LayoutCtx::packed(0).packed_offset(), Some(0));
    assert_eq!(LayoutCtx::packed(31).packed_offset(), Some(31));
}

#[test]
fn mapping_slot_matches_solidity_derivation() {
    let key = Address::repeat_byte(0x11);
    let slot = U256::from(7u64);

    let mut buf = [0u8; 64];
    buf[12..32].copy_from_slice(key.as_slice());
    buf[32..].copy_from_slice(&slot.to_be_bytes::<32>());
    let expected = U256::from_be_bytes(keccak256(buf).0);

    assert_eq!(key.mapping_slot(slot), expected);
}

/// Blanket `Storable for Packable`: a full-slot value round-trips and `delete`
/// zeroes the whole word.
#[test]
fn packable_full_roundtrip_and_delete() {
```

```

let mut slot = PackedSlot(U256::ZERO);
let addr = Address::repeat_byte(0xAB);

<Address as Storable>::store(&addr, &mut slot, U256::ZERO, LayoutCtx::FULL).unwrap();
assert_eq!(<Address as Storable>::load(&slot, U256::ZERO, LayoutCtx::FULL).unwrap(),
  ↪ addr);

<Address as Storable>::delete(&mut slot, U256::ZERO, LayoutCtx::FULL).unwrap();
assert_eq!(slot.0, U256::ZERO);
}

/// Blanket `Storable for Packable` at a byte offset: writing/deleting a packed
/// primitive must not disturb a neighbor sharing the same word.
#[test]
fn packable_packed_delete_preserves_neighbor_byte() {
  // Low byte holds a sentinel owned by a notional neighbor field.
  let mut slot = PackedSlot(U256::from(0xFFu64));

  <bool as Storable>::store(&true, &mut slot, U256::ZERO,
    ↪ LayoutCtx::packed(1)).unwrap();
  assert!(<bool as Storable>::load(&slot, U256::ZERO, LayoutCtx::packed(1)).unwrap());
  assert_eq!(slot.0 & U256::from(0xFFu64), U256::from(0xFFu64), "neighbor untouched by
    ↪ store");

  <bool as Storable>::delete(&mut slot, U256::ZERO, LayoutCtx::packed(1)).unwrap();
  assert!(!<bool as Storable>::load(&slot, U256::ZERO, LayoutCtx::packed(1)).unwrap());
  assert_eq!(slot.0 & U256::from(0xFFu64), U256::from(0xFFu64), "neighbor untouched by
    ↪ delete");
}

```

4.1.3 crates/common/precompile-macros/src/utils.rs

Coverage for slot, namespace, and array-size attribute parsing — areas that had no prior coverage.

Test	What is tested	Pass
extract_attributes_parses_decimal_and_hex_slots	<code>#[slot(42)]</code> and <code>#[slot(0x2a)]</code> resolve to the same value	✓
extract_attributes_hashes_string_slot	<code>#[slot("foo")]</code> resolves to keccak256(b"foo")	✓
extract_attributes_reads_base_slot	<code>#[base_slot(7)]</code> parsed correctly	✓
extract_attributes_rejects_duplicate_slot	Duplicate <code>#[slot]</code> attributes error	✓
extract_attributes_rejects_combined_slot_and_base_slot	<code>slot</code> + <code>base_slot</code> on the same item errors	✓
extract_attributes_rejects_combined_slot_and_namespace	<code>slot</code> + <code>namespace</code> on the same item errors	✓
array_sizes_accepts_valid_range	<code>#[storable_arrays(1, 256)]</code> accepted	✓
array_sizes_reject_zero	Size 0 rejected	✓
array_sizes_reject_over_max	Size 257 rejected	✓
array_sizes_reject_duplicates	Duplicate sizes rejected	✓
array_sizes_reject_non_integer	Non-integer size literals rejected	✓
erc7201_root_masks_low_byte_to_zero	ERC-7201 namespace root has its low byte zeroed	✓
extract_namespace_rejects_duplicate	Duplicate <code>#[namespace]</code> attributes error	✓

```

#[test]
fn extract_attributes_parses_decimal_and_hex_slots() {

```

```

    let decimal: Vec<Attribute> = vec![parse_quote!(#[slot(42)])];
    let hex: Vec<Attribute> = vec![parse_quote!(#[slot(0x2a)])];
    assert_eq!(extract_attributes(&decimal).unwrap().0, Some(U256::from(42)));
    assert_eq!(extract_attributes(&hex).unwrap().0, Some(U256::from(42)));
}

#[test]
fn extract_attributes_hashes_string_slot() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[slot("foo")])];
    let expected: U256 = keccak256(b"foo").into();
    assert_eq!(extract_attributes(&attrs).unwrap().0, Some(expected));
}

#[test]
fn extract_attributes_reads_base_slot() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[base_slot(7)])];
    assert_eq!(extract_attributes(&attrs).unwrap().1, Some(U256::from(7)));
}

#[test]
fn extract_attributes_rejects_duplicate_slot() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[slot(1)]), parse_quote!(#[slot(2)])];
    assert!(extract_attributes(&attrs).is_err());
}

#[test]
fn extract_attributes_rejects_combined_slot_and_base_slot() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[slot(1)]),
    ↪ parse_quote!(#[base_slot(2)])];
    assert!(extract_attributes(&attrs).is_err());
}

#[test]
fn extract_attributes_rejects_combined_slot_and_namespace() {
    let attrs: Vec<Attribute> =
        vec![parse_quote!(#[slot(1)]), parse_quote!(#[namespace("b20.policy")])];
    assert!(extract_attributes(&attrs).is_err());
}

#[test]
fn array_sizes_accepts_valid_range() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[storable_arrays(1, 256)])];
    assert_eq!(extract_storable_array_sizes(&attrs).unwrap(), Some(vec![1, 256]));
}

#[test]
fn array_sizes_reject_zero() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[storable_arrays(0)])];
    assert!(extract_storable_array_sizes(&attrs).is_err());
}

#[test]
fn array_sizes_reject_over_max() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[storable_arrays(257)])];
    assert!(extract_storable_array_sizes(&attrs).is_err());
}

#[test]
fn array_sizes_reject_duplicates() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[storable_arrays(4, 4)])];
    assert!(extract_storable_array_sizes(&attrs).is_err());
}

#[test]
fn array_sizes_reject_non_integer() {
    let attrs: Vec<Attribute> = vec![parse_quote!(#[storable_arrays("4")])];

```

```

    assert!(extract_storable_array_sizes(&attrs).is_err());
}

#[test]
fn erc7201_root_masks_low_byte_to_zero() {
    let id: LitStr = parse_quote!("base.token.storage");
    let root = erc7201_root(&id).unwrap();
    assert_eq!(root & U256::from(0xfu64), U256::ZERO);
}

#[test]
fn extract_namespace_rejects_duplicate() {
    let attrs: Vec<Attribute> =
        vec![parse_quote!(#[namespace("a")]), parse_quote!(#[namespace("b")])];
    assert!(extract_namespace(&attrs).is_err());
}

```

4.1.4 crates/common/precompile-macros/src/accounting.rs

A new test module for the ABI field-detection helpers used by generated accounting code (the file previously had no tests).

Test	What is tested	Pass
has_field_detects_named_field	Detects a named struct field	✓
has_field_false_for_tuple_struct	Returns false for tuple structs	✓
has_field_false_for_non_struct	Returns false for non-struct types	✓
require_field_errors_when_missing	Errors when a required field is absent	✓
require_field_ok_when_present	Succeeds when a required field is present	✓

```

#[test]
fn has_field_detects_named_field() {
    let input: DeriveInput = parse_quote! {
        struct Storage<'a> { b20: B20<'a>, asset: Asset<'a> }
    };
    assert!(has_field(&input, "b20"));
    assert!(has_field(&input, "asset"));
    assert!(!has_field(&input, "stablecoin"));
}

#[test]
fn has_field_false_for_tuple_struct() {
    let input: DeriveInput = parse_quote! { struct Storage(u8); };
    assert!(!has_field(&input, "b20"));
}

#[test]
fn has_field_false_for_non_struct() {
    let input: DeriveInput = parse_quote! { enum Storage { A } };
    assert!(!has_field(&input, "b20"));
}

#[test]
fn require_field_errors_when_missing() {
    let input: DeriveInput = parse_quote! { struct Storage { other: u8 } };
    let err = require_field(&input, "b20").unwrap_err();
    assert!(err.to_string().contains("missing `b20` field"));
}

#[test]
fn require_field_ok_when_present() {
    let input: DeriveInput = parse_quote! { struct Storage<'a> { b20: B20<'a> } };
}

```

```
    assert!(require_field(&input, "b20").is_ok());
}
```

4.1.5 crates/common/precompile-macros/src/storable.rs

Test	What is tested	Pass
validate_sequential_discriminants_rejects_too_many_variants	Enums with more than 256 variants are rejected	✓

```
#[test]
fn validate_sequential_discriminants_rejects_too_many_variants() {
    let variants =
        (0..=256).map(|index| format!("V{index}")).collect::<Vec<_>>().join(", ");
    let input: DeriveInput =
        syn::parse_str(&format!("enum TooMany {{ {variants} }}")).unwrap();
    let data_enum = parse_enum(input);
    let err = validate_sequential_discriminants(&data_enum).unwrap_err();
    assert!(err.to_string().contains("at most 256 variants"));
}
```

4.1.6 crates/common/precompile-macros/src/contract.rs

A new test module for the `#[contract]` macro front-end parsing (the file previously had no tests).

Test	What is tested	Pass
config_empty_has_no_address	An empty config yields no address	✓
config_accepts_addr_and_address	Both <code>addr =</code> and <code>address =</code> are accepted	✓
config_rejects_unknown_key	Unknown config keys error	✓
parse_fields_accepts_named_fields	Named struct fields are parsed in order	✓
parse_fields_rejects_reserved_name	Reserved names (<code>address</code> , <code>storage</code> , <code>msg_</code> sender) are rejected	✓
parse_fields_rejects_generics	Generic struct parameters are rejected	✓
parse_fields_rejects_tuple_struct	Tuple structs are rejected	✓
parse_fields_rejects_enum	Enums are rejected	✓
parse_fields_rejects_field_attrs_with_contract_namespace	Field-level <code>#[slot]</code> combined with a contract-level <code>#[namespace]</code> is rejected	✓

```
#[test]
fn config_empty_has_no_address() {
    assert!(parse_config(quote! {}).unwrap().address.is_none());
}

#[test]
fn config_accepts_addr_and_address() {
    assert!(parse_config(quote! { addr = SOME_ADDR }).unwrap().address.is_some());
    assert!(parse_config(quote! { address = SOME_ADDR }).unwrap().address.is_some());
}

#[test]
fn config_rejects_unknown_key() {
    let err = parse_config(quote! { foo = 1 }).err().unwrap();
    assert!(err.to_string().contains("only `addr` attribute is supported"));
}
```

```

#[test]
fn parse_fields_accepts_named_fields() {
    let input: DeriveInput = parse_quote! {
        struct Token { owner: Address, total_supply: U256 }
    };
    let fields = parse_fields(input, false).unwrap();
    assert_eq!(fields.len(), 2);
    assert_eq!(fields[0].name, "owner");
    assert_eq!(fields[1].name, "total_supply");
}

#[test]
fn parse_fields_rejects_reserved_name() {
    for reserved in ["address", "storage", "msg_sender"] {
        let field: syn::Ident = syn::parse_str(reserved).unwrap();
        let input: DeriveInput = parse_quote! { struct Token { #field: U256 } };
        let err = parse_fields(input, false).unwrap_err();
        assert!(err.to_string().contains("reserved"));
    }
}

#[test]
fn parse_fields_rejects_generics() {
    let input: DeriveInput = parse_quote! { struct Token<T> { value: T } };
    let err = parse_fields(input, false).unwrap_err();
    assert!(err.to_string().contains("generic"));
}

#[test]
fn parse_fields_rejects_tuple_struct() {
    let input: DeriveInput = parse_quote! { struct Token(U256); };
    assert!(parse_fields(input, false).is_err());
}

#[test]
fn parse_fields_rejects_enum() {
    let input: DeriveInput = parse_quote! { enum Token { A } };
    assert!(parse_fields(input, false).is_err());
}

#[test]
fn parse_fields_rejects_field_attrs_with_contract_namespace() {
    let input: DeriveInput = parse_quote! {
        struct Token {
            #[slot(1)]
            value: U256,
        }
    };
    let err = parse_fields(input, true).unwrap_err();
    assert!(err.to_string().contains("contract-level `namespace`"));
}

```

4.1.7 crates/common/precompile-macros/src/namespace.rs

A new test module for the `#[namespace]` attribute macro validation (the file previously had no tests).

Test	What is tested	Pass
<code>applies_to_contract_struct</code>	Expands on a <code>#[contract]</code> struct	✓
<code>applies_to_storable_struct</code>	Expands on a <code>#[derive(Storable)]</code> struct	✓

rejects_struct_without_contract_or_storable	A bare struct without pairing errors	✓
rejects_existing_namespace_attribute	A duplicate <code>#[namespace]</code> on the item errors	✓
rejects_invalid_namespace_id	Whitespace in the namespace id errors	✓
has_storable_derive_detects_storable	Detects <code>Storable</code> in the derive list	✓

```

#[test]
fn applies_to_contract_struct() {
    let attr = quote! { "b20.policy" };
    let item = quote! {
        #[contract(addr = ADDR)]
        struct S { value: U256 }
    };
    assert!(expand_impl(attr, item).is_ok());
}

#[test]
fn applies_to_storable_struct() {
    let attr = quote! { "b20.policy" };
    let item = quote! {
        #[derive(Storable)]
        struct S { value: U256 }
    };
    assert!(expand_impl(attr, item).is_ok());
}

#[test]
fn rejects_struct_without_contract_or_storable() {
    let attr = quote! { "b20.policy" };
    let item = quote! { struct S { value: U256 } };
    let err = expand_impl(attr, item).unwrap_err();
    assert!(err.to_string().contains("must be paired"));
}

#[test]
fn rejects_existing_namespace_attribute() {
    let attr = quote! { "b20.policy" };
    let item = quote! {
        #[contract(addr = ADDR)]
        #[namespace("b20.policy")]
        struct S { value: U256 }
    };
    let err = expand_impl(attr, item).unwrap_err();
    assert!(err.to_string().contains("duplicate"));
}

#[test]
fn rejects_invalid_namespace_id() {
    let attr = quote! { "has whitespace" };
    let item = quote! {
        #[contract(addr = ADDR)]
        struct S { value: U256 }
    };
    assert!(expand_impl(attr, item).is_err());
}

#[test]
fn has_storable_derive_detects_storable() {
    let with: DeriveInput = parse_quote! {
        #[derive(Debug, Storable)]
        struct S { value: U256 }
    };
    let without: DeriveInput = parse_quote! {
        #[derive(Debug, Clone)]
        struct S { value: U256 }
    };

```

```
};  
assert!(has_storable_derive(&with).unwrap());  
assert!(!has_storable_derive(&without).unwrap());  
}
```